

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



ĐỒ ÁN CHUYÊN NGÀNH
HUẤN LUYỆN DRONE NÉ VẬT CẢN DỰA TRÊN HỌC TĂNG CƯỜNG
**TRAINING DRONES FOR OBSTACLE AVOIDANCE USING
REINFORCEMENT LEARNING**

CỦ NHÂN NGÀNH MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



ĐỒ ÁN CHUYÊN NGÀNH

HUẤN LUYỆN DRONE NÉ VẬT CẢN DỰA TRÊN HỌC TĂNG CƯỜNG
TRAINING DRONES FOR OBSTACLE AVOIDANCE USING
REINFORCEMENT LEARNING

CỬ NHÂN NGÀNH MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

GIẢNG VIÊN HƯỚNG DẪN
THS. NGUYỄN KHÁNH THUẬT

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG KHÓA ĐỒ ÁN CHUYÊN NGÀNH

Hội đồng chấm đồ án chuyên ngành, thành lập theo Quyết định số
ngày của Hiệu trưởng Trường Đại học Công nghệ Thông tin.

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến thầy Nguyễn Khánh Thuật – giảng viên bộ môn “Đồ án chuyên ngành” trong Khoa Mạng Máy Tính và Truyền thông và Trường Đại học Công nghệ thông tin Thông tin – ĐHQG-HCM đã trang bị cho em những kiến thức, kỹ năng cơ bản cần có để hoàn thành đề tài nghiên cứu này.

Tuy nhiên trong quá trình nghiên cứu đề tài, do kiến thức chuyên ngành còn hạn chế nên em vẫn còn nhiều thiếu sót khi tìm hiểu, đánh giá và trình bày về đề tài. Rất mong nhận được sự quan tâm, góp ý của thầy để đề tài của nhóm được đầy đủ và hoàn chỉnh hơn.

Xin chân thành cảm ơn.

Thành phố Hồ Chí Minh, <tháng / 2026>

Sinh viên

Nguyễn Hữu Cẩm

MỤC LỤC

Lời cảm ơn	i
Mục lục	ii
Danh mục các bảng	iv
Danh mục các hình ảnh và biểu đồ	v
Danh mục từ viết tắt	vi
Tóm tắt khóa luận	vii
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI	1
1.1 Giới thiệu đề tài	1
1.2 Mục tiêu, đối tượng và phạm vi nghiên cứu	2
1.2.1 Mục tiêu nghiên cứu	2
1.2.2 Đối tượng nghiên cứu	2
1.2.3 Phạm vi nghiên cứu	2
1.3 Cấu trúc bài báo cáo	2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	4
2.1 Học tăng cường	4
2.1.1 Lịch sử phát triển của học tăng cường	4
2.1.2 Nội dung về sự tiến hóa của thuật toán DQN và sự ra đời của D3QN	6
2.2 Môi trường mô phỏng vật lý gym-pybullet-drones	8
CHƯƠNG 3. THIẾT KẾ CÁC PHẦN TRONG HỆ THỐNG	10
3.1 Sơ đồ hệ thống	10
3.2 Thiết kế môi trường	11
3.2.1 Điều chỉnh góc quan sát trong môi trường mô phỏng	11
3.2.2 Xây dựng hàm phần thưởng	12
3.2.2.1 Phần thưởng dương	12

MỤC LỤC

3.2.2.2	Phần thưởng âm	13
3.2.3	Thuật toán sinh vật cản	13
3.2.4	Góc nhìn của drone	14
3.2.5	Hệ thống cảm biến phát hiện vật cản	15
3.2.6	Vector gợi ý hành động (Action Hints)	16
3.3	Thiết kế tác tử	18
3.3.1	Bộ nhớ kinh nghiệm	18
3.3.2	Kiến trúc mạng nơ-ron	20
3.3.2.1	Nhánh xử lí ảnh	20
3.3.2.2	Nhánh xử lí vector trạng thái	21
3.3.2.3	Hợp nhất đặc trưng	22
3.3.2.4	Kiến trúc Dueling	23
3.3.2.5	Cơ chế học Double DQN	24
3.3.2.6	Cập nhật mạng mục tiêu	26
3.3.2.7	Hàm mất mát và tối ưu hóa	27
3.3.3	Các siêu tham số huấn luyện	28
CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ		29
4.1	So sánh với mô hình DQN truyền thống	29
4.2	Các bài kiểm thử khác nhau	40
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		43
5.1	Kết luận	43
5.2	Hướng phát triển	43
Tài liệu tham khảo		46
CHƯƠNG A. Phụ lục A		47
CHƯƠNG B. Phụ lục B		48

DANH MỤC CÁC BẢNG

3.1	Các siêu tham số huấn luyện	28
4.1	Các siêu tham số huấn luyện của tác tử sử dụng DQN truyền thống .	29
4.2	Kết quả đánh giá mô hình theo nguyên nhân kết thúc episode	40
4.3	Kết quả đánh giá mô hình DQN theo nguyên nhân kết thúc episode .	41
4.4	Thông kê tỷ lệ chiến thắng qua các lần kiểm thử	41

DANH MỤC CÁC HÌNH ẢNH VÀ BIỂU ĐỒ

2.1	Quy trình hoạt động của học tăng cường	5
3.1	Sơ đồ hệ thống	10
3.2	Hình ảnh về góc nhìn của trainer khi huấn luyện	12
4.1	Kết quả huấn luyện của tác tử D3QN	30
4.2	Kết quả huấn luyện của tác tử DQN	31
4.3	Biểu đồ loss của D3QN	33
4.4	Biểu đồ loss của DQN	34
4.5	Biểu đồ phân phối hành động và giá trị Q-Values của D3QN	35
4.6	Biểu đồ phân phối hành động và giá trị Q-Values của DQN	35
4.7	Biểu đồ điểm tích lũy và số bước trung bình của D3QN	38
4.8	Biểu đồ điểm tích lũy và số bước trung bình của DQN	38

DANH MỤC TỪ VIẾT TẮT

RL Học tăng cường

MNR Mạng nơ-ron

DQN Deep Q-Network

DDQN Double Deep Q-Network

D3QN Dueling Double Deep Q-Network

CNN Convolutional Neutral Network

Agent Tác tử

TÓM TẮT ĐỒ ÁN

Chương 1. TỔNG QUAN ĐỀ TÀI

1.1 Giới thiệu đề tài

Thiết bị bay không người lái là những thiết bị cơ khí có thể điều khiển linh hoạt hoặc tự vận hành, được áp dụng rộng rãi trong lĩnh vực công nghệ này nay. Chúng ta đã được biết đến rất nhiều ứng dụng của nó trong đời sống: từ vận chuyển giao nhận hàng hóa, giám sát an ninh, hỗ trợ nông nghiệp thông minh, có thể thay thế con người thực hiện các nhiệm vụ cứu hộ cứu nạn ở những địa hình hiểm trở và thấy nó nhiều trong các lĩnh vực như điện ảnh, khảo sát địa chất, hoặc robot trinh sát quân sự,...

Trong thời đại AI bùng nổ hiện nay, có thể nói các phương pháp học máy được coi là xu thế mới trong việc phát triển các công nghệ. Nổi bật trong số đó là học tăng cường, một trong ba phương pháp học máy bao gồm: Học có giám sát, học không có giám sát và học tăng cường. Học tăng cường, đặc biệt là Mạng nơ-ron Q sâu (Deep Q-Network-DQN), là việc sử dụng trí tuệ nhân tạo để giúp thiết bị tự động học cách tương tác với môi trường nhằm đạt đến các mục tiêu mong muốn. Đây là một bài toán quan trọng trong thiết kế hệ thống tự hành nói riêng và ngành trí tuệ nhân tạo trong ứng dụng nói chung. Việc áp dụng mạng nơ-ron sâu cho phép hệ thống tìm ra được các quyết định điều khiển tối ưu nhất thông qua quá trình thử và sai (trial and error) mà không cần lập trình các quy luật đường bay cứng nhắc.

Sử dụng thuật toán DQN để tính toán các cử động của drone dựa trên dữ liệu trạng thái thu thập được từ môi trường xung quanh. Điều này cho phép drone di chuyển một cách linh hoạt, độc lập và chính xác trong không gian hẹp. Ngoài ra, thuật toán còn có thể tiếp thu và tự động hóa các chuỗi hành động phức tạp giúp thiết bị nhận thức được không gian vật lý, từ đó tối ưu hóa các chuỗi hành động phức tạp giúp thiết bị nhận thức được không gian vật lý, từ đó tối ưu hóa quỹ đạo bay để tránh va chạm.

Đề tài tập trung vào việc tích hợp các yếu tố trên vào môi trường mô phỏng vật lý 3D (PyBullet), tạo ra một mô hình trí tuệ nhân tạo để điều hướng độc lập cho máy bay không người lái. Phát triển một hệ thống điều khiển thông minh sử dụng thuật toán D3QN kết hợp không gian trạng thái đa giác quan, đồng thời kết hợp với phân

1.3. CẤU TRÚC BÀI BÁO CÁO

tích dữ liệu ảnh được gắn trên Camera bằng CNN. Giúp cho drone tìm ra quỹ đạo ngắn nhất, mượt mà nhất để tránh khỏi vật cản.

1.2 Mục tiêu, đối tượng và phạm vi nghiên cứu

1.2.1 Mục tiêu nghiên cứu

Đề tài này tập trung phát triển một thông trí tuệ nhân tạo để điều hướng tự hành cho thiết bị bay không người lái (Drone) trong môi trường 3D phức tạp. Hệ thống tương tác và ra quyết định điều khiển dựa trên thuật toán Học tăng cường sâu – cụ thể là thuật toán Dueling Double Deep Q-Network (D3QN), drone sẽ liên tục thu thập trạng thái vật lý môi trường mô phỏng PyBullet. Với sự phức tạp của các bài toán định hướng không gian, hệ thống này cung cấp các quyết định điều hướng tự lái thông minh, cho phép thiết bị tự động nhận thức chướng ngại vật và tiến về đích một cách an toàn, linh hoạt mà không cần sự can thiệp của con người.

1.2.2 Đối tượng nghiên cứu

Đề tài tập trung xây dựng hệ thống mạng nơ-ron điều khiển dựa trên thuật toán D3QN. Đồng thời, nghiên cứu tiến hành thiết kế môi trường huấn luyện mô phỏng vật lý có độ chính xác cao dành cho drone, nhằm đánh giá và khẳng định tính ưu việt của giải pháp đề xuất.

1.2.3 Phạm vi nghiên cứu

Đề tài tập trung vào quá trình huấn luyện drone né vật cản dựa trên mô trường mô phỏng vật lý Gym-Pybullet-Drones

1.3 Cấu trúc bài báo cáo

Nội dung của bài báo cáo đồ án chuyên ngành sẽ bao gồm các phần sau:

- Chương 1: Giới thiệu tổng quan về đồ án.
- Chương 2: Trình bày cơ sở lý thuyết và nền tảng liên quan đến đề tài.
- Chương 3: Thiết kế các phần trong hệ thống
- Chương 4: Kết quả và đánh giá.

- Chương 5: Kết luận và hướng phát triển của đề tài

Chương 2. CƠ SỞ LÝ THUYẾT

2.1 Học tăng cường

2.1.1 Lịch sử phát triển của học tăng cường

Học tăng cường là một nhánh của học máy tập trung vào bài toán ra quyết định tuần tự. Lịch sử hình thành của RL bắt nguồn từ hai dòng nguyên cứu độc lập: lý thuyết điều khiển tối ưu (Optimal Control) trong toán học ứng dụng và lý thuyết điều kiện hóa hành vi (Behavioral Psychology) trong tâm lý học. Vào những năm 1950, Richard Bellman đặt nền tảng toán học cho lĩnh vực này thông qua phương trình quy hoạch động (Dynamic Programming) [1], cung cấp công cụ tính toán chính thức cho các bài toán tối ưu hóa theo chuỗi thời gian. Khung lý thuyết quyết định Markov (Markov Decision Process - MDP) sau đó được Howard (1960) hệ thống hóa [2], tạo mô hình hóa chuẩn mực cho môi trường RL. Tuy nhiên, phải đến thập niên 1980-1990, các thuật toán học trực tuyến như Temporal Difference Learning (Sutton, 1988) [3] và Q-Learning (Watkins, 1989) [4] mới định hình RL thành một lĩnh vực nghiên cứu độc lập, cho phép tác tử học hỏi thông qua các tương tác trực tiếp với môi trường mà không yêu cầu dữ liệu huấn luyện có nhãn.

Một bài toán học tăng cường (Reinforcement Learning - RL) thường bao gồm các thành phần cơ bản sau:

- Tác tử (Agent): Là thực thể thực hiện các hành động trong môi trường nhằm đạt được mục tiêu mà bài toán đặt ra. Thông qua quá trình tương tác và học hỏi từ các phản hồi của môi trường, tác tử dần cải thiện chiến lược ra quyết định của mình.
- Môi trường (Environment): Là không gian mà tác tử hoạt động và tương tác. Môi trường tiếp nhận các hành động từ tác tử, thay đổi trạng thái tương ứng và trả về các phản hồi để tác tử tiếp tục học tập.
- Hành động (Action): Là thao tác hoặc quyết định mà tác tử thực hiện tại một thời điểm nhất định nhằm tác động lên môi trường.
- Trạng thái (State): Là tập hợp thông tin mô tả tình huống hiện tại của môi trường hoặc của tác tử tại một thời điểm cụ thể. Trạng thái là cơ sở để tác tử đưa ra quyết định hành động tiếp theo.
- Phần thưởng (Reward): Là tín hiệu phản hồi mà tác tử nhận được từ môi trường

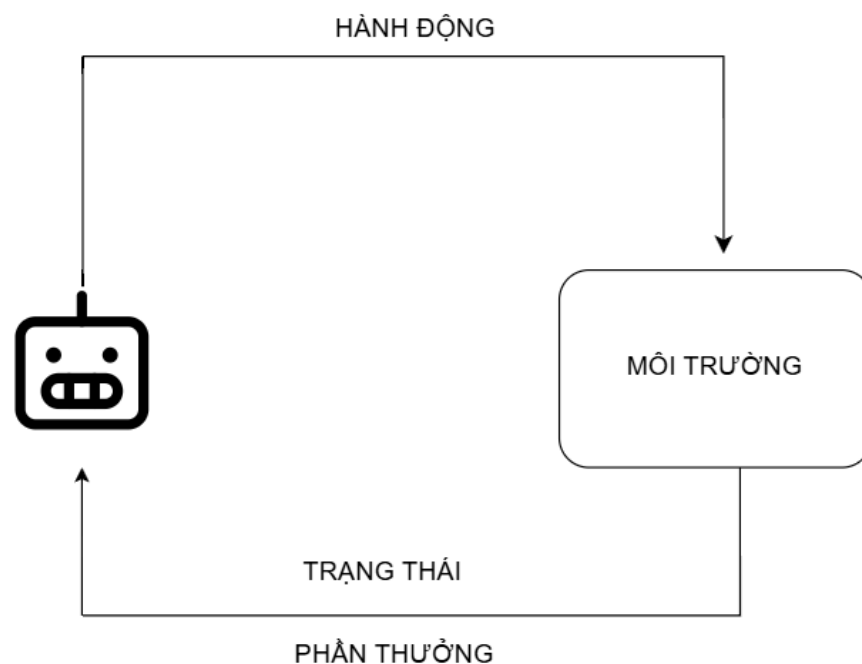
sau khi thực hiện một hành động. Giá trị phần thưởng phản ánh mức độ tốt hoặc xấu của hành động đó đối với mục tiêu cần đạt được.

- Episode: Là một quá trình tương tác hoàn chỉnh giữa tác tử và môi trường, bắt đầu từ trạng thái khởi tạo và kết thúc khi đạt mục tiêu hoặc thỏa mãn điều kiện dừng của môi trường.

- Phần thưởng tích lũy (Cumulative Reward): Là tổng các phần thưởng mà tác tử nhận được trong suốt một episode. Đây là một trong những chỉ số quan trọng để đánh giá hiệu quả của chính sách học được.

Quá trình học tăng cường diễn ra thông qua sự tương tác liên tục giữa tác tử và môi trường. Ban đầu, tác tử quan sát trạng thái hiện tại của môi trường và lựa chọn một hành động dựa trên chính sách hiện có. Sau khi hành động được thực hiện, môi trường sẽ chuyển sang trạng thái mới và trả về một phần thưởng tương ứng. Dựa trên trạng thái mới và phần thưởng nhận được, tác tử sẽ đánh giá hiệu quả của hành động vừa thực hiện và điều chỉnh chính sách nhằm đưa ra các quyết định tốt hơn trong tương lai.

Quá trình này được lặp lại nhiều lần qua các episode khác nhau. Mục tiêu cuối cùng của tác tử là học được một chính sách tối ưu giúp tối đa hóa tổng phần thưởng tích lũy trong suốt quá trình tương tác với môi trường.



Hình 2.1. Quy trình hoạt động của học tăng cường

2.1.2 Nội dung về sự tiến hóa của thuật toán DQN và sự ra đời của D3QN

DQN là thuật toán nền tảng của học tăng cường sâu, được Mnih và cộng sự (DeepMind) công bố năm 2013 và hoàn thiện năm 2015 [5], [6]. Ý tưởng cốt lõi là thay thế bảng Q-Table truyền thống bằng một mạng nơ-ron sâu đóng vai trò hàm xấp xỉ $Q(s, a; \theta)$, trong đó θ là trọng số mạng. Với không gian trạng thái liên tục hoặc có chiều cao (như ảnh pixel), bảng Q-Table không thể lưu trữ mọi cặp (s, a) ; mạng nơ-ron giải quyết vấn đề này bằng khả năng tổng quát hóa [6], [7].

DQN sử dụng hai kỹ thuật ổn định hóa huấn luyện quan trọng: Experience Replay (lưu lại lịch sử chuyển trạng thái (s, a, r, s') vào bộ nhớ và lấy mẫu ngẫu nhiên khi cập nhật, phá vỡ sự tương quan của tuần tự dữ liệu) và Target Network (một bản sao có trọng số được cập nhật chậm hơn, dùng để tính nhân huấn luyện ổn định hơn) [6]. Quy tắc cập nhật trọng số dựa trên hàm mất mát:

$$L(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

Trong đó:

- r : phần thưởng nhận được.
- γ : hệ số chiết khấu (discount factor).
- θ : tham số của mạng hiện tại.
- θ^- : tham số của mạng mục tiêu (target network).
- $\max_{a'} Q(s', a'; \theta^-)$: giá trị Q lớn nhất ở trạng thái kế tiếp.

Tuy nhiên, hạn chế cố hữu của DQN là toán tử $\max_{a'}$ được áp dụng lên cùng một mạng $Q(s', a'; \theta^-)$ để vừa *chọn* hành động tốt nhất vừa *đánh giá* giá trị của nó. Khi các ước lượng Q còn nhiều (đặc biệt ở giai đoạn đầu huấn luyện), việc luôn chọn giá trị cực đại dẫn đến hiện tượng ước lượng thổi phồng có hệ thống (*overestimation bias*), làm chậm hội tụ và đôi khi gây mất ổn định. Để giải quyết vấn đề đó Van Hasselt và cộng sự đã đưa ra thuật toán mới Double Deep Q-Learning (DDQN) [8]. DDQN tách rời nhiệm vụ bằng cách khai thác các mạng trong DQN theo cách khác nhau:

- Online network θ : chọn hành động tối ưu

$$a^* = \arg \max_{a'} Q(s', a'; \theta)$$

- Target network θ^- : đánh giá giá trị của hành động đã chọn

$$Q(s', a^*; \theta^-)$$

Công thức toán học của mục tiêu DDQN được định nghĩa như sau:

$$Y^{DDQN} = R_{t+1} + \gamma Q\left(s_{t+1}, \arg \max_{a'} Q(s_{t+1}, a'; \theta); \theta^-\right)$$

Bên cạnh bằng việc tối ưu bằng DDQN, các nhà nghiên cứu nhận ra rằng cấu trúc mạng nơ-ron phẳng của DQN mang tính dư thừa về mặt tính toán. Năm 2016, Wang và các cộng sự giới thiệu Dueling Network Architecture [9]. Dueling DQN phân rã hàm giá trị $Q(s,a)$ thành hai thành phần độc lập: Hàm giá trị trạng thái $V(s)$ (đánh giá trạng thái đó tốt hay xấu một cách tổng quát) và hàm lợi thế $A(s,a)$ (đánh giá mức độ vượt trội của một hành động cụ thể so với các lựa chọn khác). Công thức tổng quát của Dueling DQN là:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a'; \theta, \alpha) \right)$$

- $Q(s, a)$: giá trị hành động tổng thể.
- $V(s)$: giá trị của trạng thái.
- $A(s, a)$: lợi thế (advantage) của hành động a tại trạng thái s .
- $\mathcal{A}$: tập các hành động có thể thực hiện.
- θ : tham số chung của mạng.
- α : tham số của nhánh advantage.
- β : tham số của nhánh value.

Từ những tối ưu của hai thuật toán ở trên, người ta đã kết hợp và tạo ra D3QN. Trong cấu trúc D3QN, sự lựa chọn hành động tối ưu cho trạng thái tiếp theo được xác định bởi kiến trúc dueling của mạng chính θ :

$$a^* = \arg \max_{a'} \left(V(s'; \theta) + A(s', a'; \theta) - \frac{1}{|\mathcal{A}|} \sum_{a'' \in \mathcal{A}} A(s', a''; \theta) \right)$$

Sau đó, việc ước lượng giá trị mục tiêu TD được thực hiện bởi mạng mục tiêu dueling θ^- :

$$\hat{Q}_{target} = V(s'; \theta^-) + A(s', a^*; \theta^-) - \frac{1}{|\mathcal{A}|} \sum_{a'' \in \mathcal{A}} A(s', a''; \theta^-)$$

2.2 Môi trường mô phỏng vật lý gym-pybullet-drones

Gym-pybullet-drones là một môi trường mô phỏng mã nguồn mở được phát triển bởi Panerati và cộng sự tại Đại học Toronto (2021), được thiết kế chuyên biệt cho bài toán học tăng cường ứng dụng trên thiết bị bay không người lái (UAV) [10]. Môi trường này được xây dựng trên nền tảng PyBullet — một công cụ mô phỏng vật lý dựa trên thư viện Bullet Physics Engine, cung cấp khả năng mô phỏng động lực học bay với độ trung thực cao trong không gian ba chiều [11].

Điểm nổi bật của gym-pybullet-drones là khả năng tích hợp liền mạch với giao diện chuẩn OpenAI Gym, cho phép các thuật toán học tăng cường tương tác với môi trường thông qua vòng lặp quan sát — hành động — phần thưởng quen thuộc [10], [12]. Môi trường hỗ trợ mô phỏng nhiều loại UAV khác nhau, trong đó mô hình Crazyflie 2.x được sử dụng phổ biến nhất nhờ tính tương đồng cao với phần cứng thực tế. Động lực học của UAV được tính toán dựa trên các phương trình vật lý thực, bao gồm lực nâng, lực cản không khí và mô men xoắn, giúp chính sách học được trong môi trường ảo có khả năng chuyển giao (sim-to-real transfer) tốt hơn so với các mô phỏng đơn giản hóa.

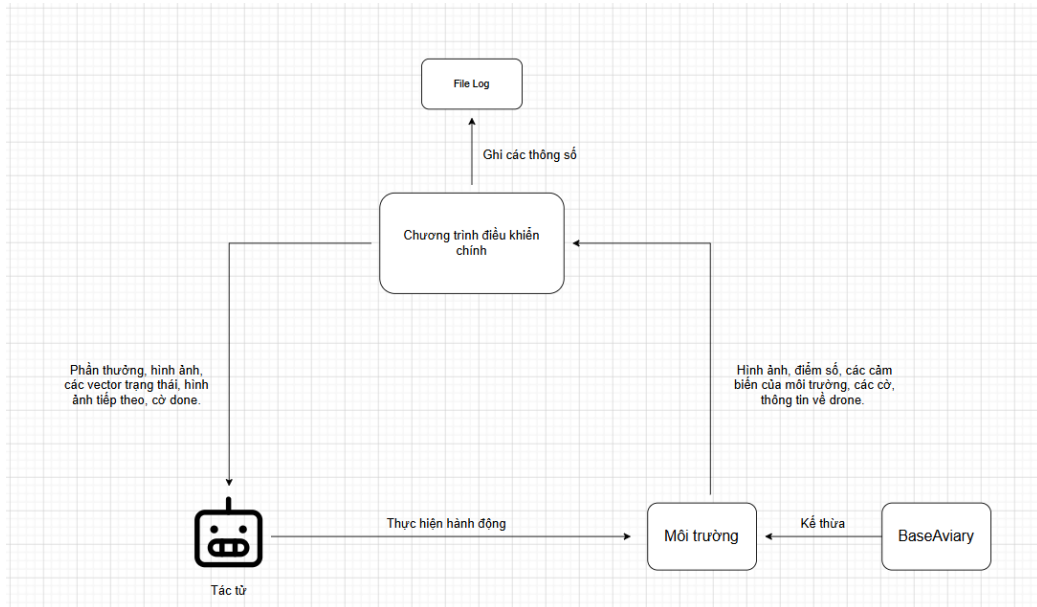
Về cấu trúc môi trường, gym-pybullet-drones cung cấp nhiều chế độ hoạt động khác nhau phù hợp với từng mục tiêu nghiên cứu, từ điều khiển cấp thấp (điều chỉnh tốc độ từng rotor) đến điều khiển cấp cao (đặt mục tiêu vị trí và vận tốc) [10]. Môi

trường cũng hỗ trợ tùy chỉnh không gian quan sát, bao gồm thông tin về vị trí, vận tốc, góc nghiêng của UAV, đồng thời cho phép bổ sung các vật cản tĩnh vào không gian làm việc, tạo điều kiện huấn luyện cho các bài toán tránh vật cản phức tạp.

Trong khuôn khổ đề án này, gym-pybullet-drones được lựa chọn làm môi trường huấn luyện chính nhờ sự cân bằng giữa độ chính xác vật lý và hiệu năng tính toán, cho phép tiến hành hàng triệu bước mô phỏng trong thời gian hợp lý mà không yêu cầu phần cứng chuyên biệt. Đây là nền tảng phù hợp để đánh giá hiệu quả của thuật toán D3QN trong bài toán điều hướng UAV tự động.

Chương 3. THIẾT KẾ CÁC PHẦN TRONG HỆ THỐNG

3.1 Sơ đồ hệ thống



Hình 3.1. Sơ đồ hệ thống

Hệ thống huấn luyện được xây dựng dựa trên ba thành phần chính gồm môi trường, tác tử và chương trình điều khiển chính. Ba thành phần này phối hợp với nhau để tạo thành một vòng lặp học tăng cường hoàn chỉnh.

- **Môi trường (Environment):** Được xây dựng dựa trên lớp `BaseAviary` của thư viện `gym-pybullet-drones`. Thành phần này chịu trách nhiệm mô phỏng quá trình hoạt động của UAV trong không gian ba chiều, quản lý các vật cản, xác định vị trí mục tiêu và cung cấp các thông tin quan sát cho tác tử. Các dữ liệu được môi trường trả về bao gồm trạng thái động học của UAV, dữ liệu cảm biến khoảng cách, hình ảnh quan sát từ camera và các tín hiệu phần thưởng tương ứng với từng hành động.
- **Tác tử (Agent):** Là thành phần thực hiện quá trình học tập và ra quyết định. Tác tử tiếp nhận ảnh quan sát và vector trạng thái từ môi trường, sau đó sử dụng mô hình D3QN để lựa chọn hành động phù hợp. Ngoài ra, tác tử còn quản lý bộ nhớ `Experience Replay`, lưu trữ các kinh nghiệm thu thập được trong quá

trình tương tác và thực hiện cập nhật trọng số mạng nơ-ron nhằm tối ưu hóa chính sách điều khiển.

- Chương trình điều khiển chính (Main Controller): Đóng vai trò trung gian kết nối giữa tác tử và môi trường. Thành phần này chịu trách nhiệm khởi tạo các đối tượng cần thiết, xử lý dữ liệu đầu vào trước khi gửi tới tác tử, thực hiện vòng lặp huấn luyện hoặc đánh giá, ghi nhận các thông tin thống kê, lưu trữ mô hình và quản lý toàn bộ quá trình thực thi của hệ thống. Nhờ đó, quá trình huấn luyện được diễn ra ổn định, đồng bộ và thuận lợi cho việc theo dõi kết quả thực nghiệm.

3.2 Thiết kế môi trường

Đồ án này xây dựng môi trường kế thừa môi trường gốc của gym-pybullet-drones là BaseAviary. Đây là môi trường được tác giả viết ra, các tương tác tương tự như ngoài đời đều được tác giả mô phỏng sát với thực tế.

3.2.1 Điều chỉnh góc quan sát trong môi trường mô phỏng

Để có một góc nhìn phù hợp để quan sát drone, chúng ta cần thay đổi.

Tại dòng 153 đến dòng 158:

```
p.resetDebugVisualizerCamera(  
    cameraDistance=3,  
    cameraYaw=-30,  
    cameraPitch=-30,  
    cameraTargetPosition=[0, 0, 0],  
    physicsClientId=self.CLIENT  
)
```

Thành đoạn code sau đây:

```
p.resetDebugVisualizerCamera(  
    cameraDistance=20,
```

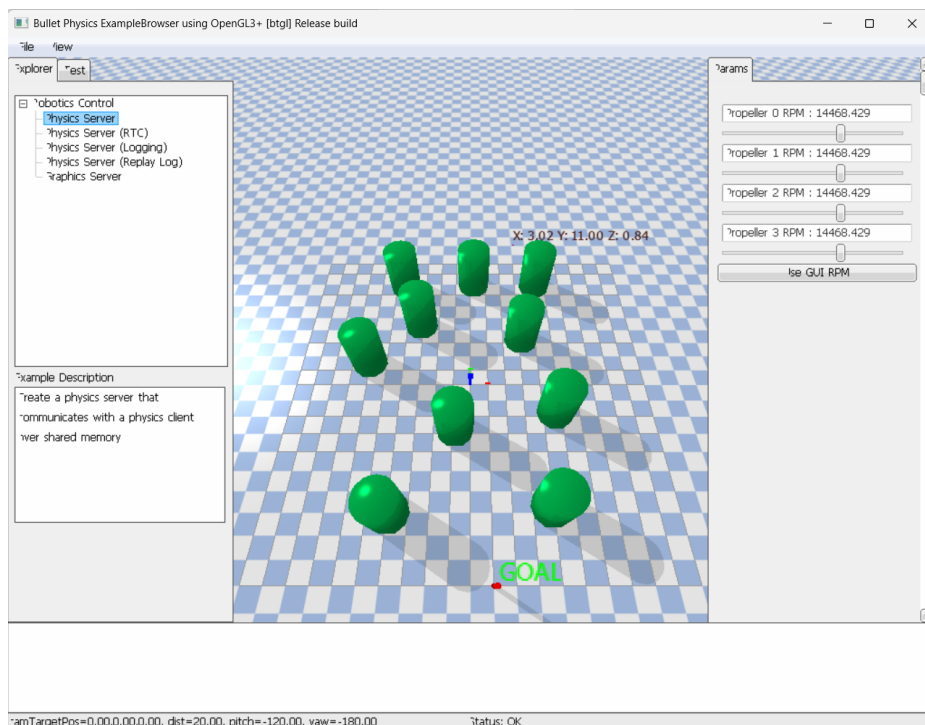
3.2. THIẾT KẾ MÔI TRƯỜNG

```
cameraYaw=-180,  
cameraPitch=-120,  
cameraTargetPosition=[0, 0, 0],  
physicsClientId=self.CLIENT  
)
```

Đoạn code trên thay đổi góc nhìn từ:

- Khoảng cách từ 3 đơn vị sang 20 đơn vị.
- Thay đổi góc Yaw từ -30 độ sang -180 độ.
- Thay đổi góc Pitch từ -30 độ sang -120 độ.

Lưu ý: Tất các thay đổi ở trên đều không ảnh hưởng đến kết quả của đề án. Đây chỉ là những thay đổi về góc nhìn đề án. Các góc ở đây có thể thay đổi theo cảm nhận của từng người.



Hình 3.2. Hình ảnh về góc nhìn của trainer khi huấn luyện

3.2.2 Xây dựng hàm phần thưởng

3.2.2.1 Phần thưởng dương

Ở môi trường này, tác tử sẽ có 3 cách để nhận được phần thưởng dương:

- Phần thưởng khi tới gần mục tiêu: Công thức sẽ là

$$R_{\text{progress}} = ((d_{\text{prev}} - d_{\text{curr}}) \times 2.0) - 0,001$$

Trong đó:

- d_{prev} : khoảng cách từ UAV đến mục tiêu ở bước trước.
- d_{curr} : khoảng cách từ UAV đến mục tiêu ở bước hiện tại.

- Thưởng khi tới đích: Môi trường sẽ thưởng cho tác tử +500 điểm ứng với mỗi lần tác tử tới gần đích trong bán kính 1 đơn vị tính từ đích.

3.2.2.2 Phần thưởng âm

Bên cạnh các phần thưởng nhằm khuyến khích tác tử thực hiện những hành vi mong muốn, môi trường cũng áp dụng các hình phạt đối với những hành vi không mong muốn nhằm định hướng quá trình học hiệu quả hơn. Các hình phạt bao gồm:

- Phạt khi đụng cột: Đây là hình phạt khi mà drone di chuyển và đụng phải cột. Đây là cách để tác tử học được cách di chuyển sao cho tránh được vật cản. Với mỗi lần vi phạm tác tử sẽ bị -200 điểm.

- Phạt khi vượt quá giới hạn: Môi trường được thiết kế là hình chữ nhật có diện tích là 13x24 đơn vị với $x \in (-6.5, 6.5)$ và $y \in (-12, 12)$. Mỗi lần vi phạm lỗi này, tác tử sẽ bị -200 điểm.

- Phạt đi quá số bước: Với mỗi episode mà tác tử tốn quá nhiều bước để tới đích để di chuyển thì tác tử sẽ bị -200 điểm.

- Các lỗi trên mỗi lần vi phạm màn chơi sẽ kết thúc và quay lại trạng thái ban đầu.

3.2.3 Thuật toán sinh vật cản

Bên cạnh việc thiết kế hàm phần thưởng, việc xây dựng môi trường huấn luyện, bao gồm vật cản, vị trí xuất phát và vị trí đích, cũng đóng vai trò quan trọng đối với hiệu quả học của tác tử. Trong đồ án này, các vị trí được định nghĩa sẵn trong tệp

cấu hình (*.config*), đóng vai trò là tập hợp các điểm có thể được lựa chọn làm vị trí xuất phát hoặc vị trí đích của UAV.

Để tăng tính đa dạng của môi trường, 8 trong số 10 vị trí được sử dụng kết hợp với một độ lệch ngẫu nhiên trong quá trình khởi tạo. Cơ chế này giúp tạo ra nhiều kịch bản huấn luyện khác nhau, tránh việc UAV chỉ học thuộc một số tuyến đường cố định. Nhờ đó, mô hình có khả năng tổng quát hóa tốt hơn và giảm nguy cơ xảy ra hiện tượng overfitting đối với các vị trí đã xuất hiện trong quá trình huấn luyện.

3.2.4 Góc nhìn của drone

Bên cạnh việc xây dựng môi trường huấn luyện và thiết kế hàm phần thưởng, việc xác định góc nhìn quan sát của UAV cũng là một yếu tố quan trọng ảnh hưởng trực tiếp đến hiệu quả học của tác tử. Trong bài toán điều hướng dựa trên ảnh, dữ liệu đầu vào của mạng nơ-ron được lấy từ camera gắn trên UAV. Do đó, chất lượng và tính nhất quán của góc nhìn sẽ quyết định khả năng nhận biết môi trường cũng như đưa ra hành động phù hợp của tác tử.

Trong đồ án này, camera được gắn trực tiếp trên UAV và được đặt tại vị trí của drone với một độ lệch nhỏ theo trục (Z) nhằm mô phỏng góc nhìn thực tế của cảm biến. Thay vì cho phép camera xoay tự do theo hướng chuyển động, góc nhìn được cố định theo hướng âm của trục (Y). Điều này xuất phát từ đặc thù của bài toán, trong đó UAV chỉ có nhiệm vụ di chuyển từ phía trên xuống phía dưới của môi trường.

Việc cố định hướng quan sát mang lại nhiều lợi ích trong quá trình huấn luyện. Thứ nhất, hình ảnh đầu vào luôn được thu nhận dưới cùng một hệ quy chiếu, giúp giảm sự biến thiên không cần thiết của dữ liệu quan sát. Thứ hai, tác tử không phải đồng thời học cả nhiệm vụ điều hướng và nhiệm vụ thích nghi với các góc nhìn khác nhau, từ đó giảm độ phức tạp của không gian trạng thái. Cuối cùng, việc duy trì góc nhìn nhất quán giúp mạng nơ-ron tập trung vào việc nhận biết vật cản, vị trí mục tiêu và các đặc trưng quan trọng của môi trường, góp phần tăng tốc độ hội tụ và cải thiện độ ổn định của quá trình huấn luyện.

Mặc dù cách tiếp cận này làm giảm tính tổng quát của mô hình đối với các kịch bản bay tự do trong không gian ba chiều, nó phù hợp với mục tiêu của đồ án và giúp tập trung đánh giá hiệu quả của thuật toán điều hướng trong một môi trường có cấu

trúc xác định.

3.2.5 Hệ thống cảm biến phát hiện vật cản

Để giúp UAV nhận biết các vật cản trong môi trường, đồ án sử dụng cơ chế cảm biến khoảng cách dựa trên kỹ thuật *ray casting* được cung cấp bởi PyBullet. Phương pháp này mô phỏng nguyên lý hoạt động của cảm biến khoảng cách bằng cách phát ra các tia dò từ vị trí hiện tại của UAV và kiểm tra xem các tia đó có va chạm với vật thể nào trong môi trường hay không.

Trong hệ thống được xây dựng, UAV phát đồng thời bốn tia dò với khoảng cách tối đa là 5 đơn vị. Các tia được bố trí theo hai nhóm:

- Hai tia chéo tạo với phương chuyển động một góc 45° , bao gồm:
 - Tia chéo trái.
 - Tia chéo phải.
- Hai tia ngang tạo góc 90° , bao gồm:
 - Tia ngang trái.
 - Tia ngang phải.

Cấu hình này cho phép UAV quan sát không gian phía trước và hai bên, từ đó phát hiện sớm các vật cản có khả năng gây va chạm. Hình thức bố trí cảm biến được minh họa như sau:

Chéo trái (45°) Chéo phải (45°)

Trái (90°) Phải (90°)

Mỗi tia được thực hiện thông qua hàm `rayTest()` của PyBullet. Hàm này trả về hệ số va chạm chuẩn hóa trong khoảng từ 0 đến 1. Nếu tia không phát hiện vật cản trong phạm vi tối đa thì giá trị nhận được bằng 1. Ngược lại, khi vật cản xuất hiện gần UAV, giá trị trả về sẽ nhỏ hơn 1 và giảm dần theo khoảng cách tới vật cản.

Giả sử d là khoảng cách từ UAV tới vật cản gần nhất trên hướng của tia dò và d_{max} là tầm phát hiện tối đa của cảm biến, giá trị chuẩn hóa được xác định theo công thức:

$$Sensor = \frac{d}{d_{max}}$$

Trong đó:

- $Sensor$: giá trị cảm biến sau khi chuẩn hóa.
- d : khoảng cách thực tế tới vật cản.
- d_{max} : khoảng cách phát hiện tối đa của tia dò.

Kết quả cuối cùng được biểu diễn dưới dạng vector cảm biến gồm bốn phần tử:

$$S_{ray} = [s_{left_diag}, s_{right_diag}, s_{left}, s_{right}]$$

Trong đó các phần tử lần lượt biểu diễn khoảng cách chuẩn hóa theo hướng chéo trái, chéo phải, trái và phải. Vector này được đưa trực tiếp vào không gian trạng thái của tác tử học tăng cường, giúp UAV đánh giá mức độ an toàn của môi trường xung quanh trước khi lựa chọn hành động tiếp theo.

3.2.6 Vector gợi ý hành động (Action Hints)

Bên cạnh các thông tin về trạng thái động học và cảm biến môi trường, đồ án còn bổ sung một vector đặc trưng gọi là *Action Hints*. Thành phần này cung cấp cho tác tử thông tin định hướng về mức độ hiệu quả của từng hành động đối với mục tiêu cuối cùng là di chuyển tới đích.

Tại mỗi thời điểm, hệ thống giả lập trước vị trí UAV sau khi thực hiện từng hành động khả thi. Đối với mỗi vị trí dự đoán, khoảng cách từ vị trí đó đến mục tiêu được tính toán và so sánh với khoảng cách hiện tại của UAV tới mục tiêu.

Giả sử khoảng cách hiện tại từ UAV đến mục tiêu là:

$$d_{current} = \|p_{current} - p_{goal}\|$$

và khoảng cách từ vị trí dự đoán của hành động thứ i đến mục tiêu là:

$$d_i = \|p_i - p_{goal}\|$$

Trong đó:

- $p_{current}$: vị trí hiện tại của UAV.
- p_i : vị trí dự đoán sau khi thực hiện hành động thứ i .
- p_{goal} : vị trí mục tiêu.

Giá trị gợi ý của hành động thứ i được xác định bởi:

$$h_i = d_{current} - d_i$$

Trong đó:

- $d_{current}$: khoảng cách hiện tại từ UAV đến mục tiêu.
- d_i : khoảng cách từ vị trí dự đoán của hành động thứ i đến mục tiêu.
- h_i : mức độ cải thiện khoảng cách của hành động thứ i .

Nếu $h_i > 0$ thì hành động đó giúp UAV tiến gần hơn tới mục tiêu. Ngược lại, nếu $h_i < 0$ thì hành động sẽ làm UAV di chuyển xa mục tiêu hơn.

Sau khi tính toán cho toàn bộ tập hành động, các giá trị được chuẩn hóa nhằm giới hạn miền giá trị và tránh ảnh hưởng quá lớn đến quá trình học. Giá trị chuẩn hóa được xác định theo công thức:

$$h'_i = \frac{h_i}{\max(|h|)} \times 0.3$$

Trong đó:

- h_i : giá trị gợi ý ban đầu.
- $\max(|h|)$: giá trị tuyệt đối lớn nhất trong vector gợi ý.
- h'_i : giá trị sau khi chuẩn hóa.

Đối với bài toán này, UAV có 5 hành động điều khiển nên vector Action Hints được biểu diễn bởi:

$$H = [h_1, h_2, h_3, h_4, h_5]$$

Vector này không được sử dụng để quyết định hành động trực tiếp mà đóng vai trò như một nguồn thông tin bổ sung cho mạng nơ-ron. Nhờ đó, tác tử có thể học nhanh hơn trong giai đoạn đầu huấn luyện do đã có thêm thông tin về xu hướng di chuyển nào giúp tiến gần tới mục tiêu.

3.3 Thiết kế tác tử

3.3.1 Bộ nhớ kinh nghiệm

Trong học tăng cường, các mẫu dữ liệu được sinh ra liên tiếp trong quá trình tương tác giữa tác tử và môi trường thường có tính tương quan rất cao. Nếu sử dụng trực tiếp các mẫu này để huấn luyện mạng nơ-ron, mô hình có thể học không ổn định, dễ dao động và khó hội tụ đến nghiệm tối ưu. Để khắc phục vấn đề này, thuật toán DQN và các biến thể của nó sử dụng cơ chế bộ nhớ kinh nghiệm (Replay Buffer).

Replay Buffer là một vùng lưu trữ dùng để ghi nhận các kinh nghiệm mà tác tử thu thập được trong quá trình tương tác với môi trường. Mỗi kinh nghiệm được biểu diễn dưới dạng một bộ dữ liệu gồm trạng thái hiện tại, hành động được thực hiện, phần thưởng nhận được, trạng thái kế tiếp và trạng thái kết thúc tập huấn luyện:

Đối với bài toán điều hướng UAV trong đồ án này, mỗi kinh nghiệm được lưu trữ dưới dạng:

$$(s_{\text{img}}, s_{\text{vec}}, a, r, s'_{\text{img}}, s'_{\text{vec}}, done)$$

Trong đó:

- s_{img} : ảnh quan sát hiện tại từ camera UAV.
- s_{vec} : vector trạng thái hiện tại của UAV.
- a : hành động được lựa chọn.

- r : phần thưởng nhận được.
- s'_{img} : ảnh quan sát ở trạng thái kế tiếp.
- s'_{vec} : vector trạng thái ở trạng thái kế tiếp.
- $done$: biến đánh dấu episode đã kết thúc hay chưa.

Đối với bài toán điều hướng UAV trong đề án này, mỗi trạng thái được cấu thành từ hai thành phần chính gồm ảnh quan sát thu được từ camera gắn trên UAV và vector trạng thái chứa các thông tin động học của thiết bị bay. Các kinh nghiệm thu thập được sẽ được lưu trữ trong bộ nhớ với dung lượng tối đa 50.000 mẫu để phục vụ quá trình huấn luyện.

Các kinh nghiệm được sinh ra trong quá trình tác tử tương tác với môi trường thường có tính tương quan rất cao do chúng xuất hiện liên tiếp theo thời gian. Nếu sử dụng trực tiếp các mẫu này để huấn luyện mạng nơ-ron, mô hình có thể học lệch theo các chuỗi trạng thái ngắn hạn, dẫn đến hiện tượng dao động hoặc mất ổn định trong quá trình hội tụ.

Để giải quyết vấn đề này, Replay Buffer áp dụng cơ chế lấy mẫu ngẫu nhiên (Random Sampling). Thay vì sử dụng các kinh nghiệm theo đúng thứ tự thu thập được, một tập dữ liệu huấn luyện có kích thước cố định (mini-batch) sẽ được lựa chọn ngẫu nhiên từ bộ nhớ kinh nghiệm.

Các kinh nghiệm được lấy mẫu ngẫu nhiên theo dạng:

$$Batch = \{e_1, e_2, \dots, e_N\}$$

Trong đó:

- e_i : một kinh nghiệm bất kỳ được lưu trữ trong Replay Buffer.
- N : kích thước mini-batch sử dụng trong một lần cập nhật mạng nơ-ron.

Việc lấy mẫu ngẫu nhiên giúp phá vỡ tính tương quan giữa các kinh nghiệm liên tiếp, từ đó làm cho dữ liệu huấn luyện gần với giả định độc lập và phân phối đồng nhất hơn. Điều này giúp quá trình cập nhật trọng số của mạng nơ-ron trở nên ổn

định hơn, giảm nguy cơ hội tụ đến nghiệm cục bộ và nâng cao hiệu quả học của tác tử.

Trong đồ án này, mỗi lần huấn luyện tác tử sẽ lấy ngẫu nhiên 64 kinh nghiệm từ Replay Buffer để tạo thành một mini-batch phục vụ quá trình tính toán hàm mất mát và cập nhật trọng số mạng.

3.3.2 Kiến trúc mạng nơ-ron

3.3.2.1 Nhánh xử lý ảnh

Để giúp tác tử nhận biết môi trường xung quanh, UAV được trang bị một camera phía trước dùng để thu thập hình ảnh quan sát trong quá trình bay. Dữ liệu ảnh chứa nhiều thông tin quan trọng như vị trí vật cản, khoảng không gian có thể di chuyển và hướng tiến về mục tiêu. Tuy nhiên, dữ liệu ảnh có kích thước lớn và mang tính không gian cao nên không thể đưa trực tiếp vào các lớp kết nối đầy đủ (Fully Connected Layer). Vì vậy, đồ án sử dụng mạng nơ-ron tích chập (Convolutional Neural Network - CNN) để trích xuất các đặc trưng quan trọng từ ảnh đầu vào.

Nhánh xử lý ảnh của mô hình bao gồm ba lớp tích chập liên tiếp, mỗi lớp được theo sau bởi hàm kích hoạt ReLU. Các lớp tích chập có nhiệm vụ tự động học các đặc trưng của môi trường ở nhiều mức độ khác nhau, từ các cạnh đơn giản đến các cấu trúc phức tạp như vật cản hoặc hành lang di chuyển.

Kiến trúc của nhánh xử lý ảnh được xây dựng như sau:

- Lớp tích chập thứ nhất: 16 bộ lọc kích thước 3×3 , bước nhảy (stride) bằng 2.
- Lớp tích chập thứ hai: 32 bộ lọc kích thước 3×3 , bước nhảy bằng 2.
- Lớp tích chập thứ ba: 64 bộ lọc kích thước 3×3 , bước nhảy bằng 2.
- Hàm kích hoạt ReLU được sử dụng sau mỗi lớp tích chập.
- Lớp Flatten được sử dụng để chuyển đổi các đặc trưng hai chiều thành vector một chiều.

Sau khi đi qua ba lớp tích chập, ảnh đầu vào được chuyển đổi thành một vector đặc trưng có kích thước 3072 phần tử. Vector này chứa các thông tin đã được trích

xuất từ môi trường và sẽ được kết hợp với các thông tin trạng thái của UAV ở các bước xử lý tiếp theo.

3.3.2.2 Nhánh xử lý vector trạng thái

Bên cạnh dữ liệu ảnh thu được từ camera, UAV còn sở hữu nhiều thông tin trạng thái quan trọng như vị trí, vận tốc, góc quay, dữ liệu cảm biến khoảng cách và các thông số động học khác. Những thông tin này được biểu diễn dưới dạng một vector trạng thái có kích thước 21 phần tử và được sử dụng như nguồn dữ liệu bổ sung giúp tác tử hiểu rõ hơn về trạng thái hiện tại của bản thân cũng như môi trường xung quanh.

Cụ thể, vector trạng thái bao gồm thông tin về vị trí UAV, góc Euler, vận tốc tuyến tính, vận tốc góc, các giá trị Action Hints và dữ liệu thu được từ hệ thống cảm biến ray casting. Việc kết hợp các thông tin này giúp tác tử không chỉ quan sát được môi trường thông qua ảnh đầu vào mà còn nắm bắt được trạng thái động học hiện tại của UAV.

Do dữ liệu trạng thái đã được biểu diễn dưới dạng vector số, đồ án sử dụng mạng nơ-ron kết nối đầy đủ (Fully Connected Network) để thực hiện quá trình trích xuất đặc trưng. Nhánh xử lý vector trạng thái bao gồm hai lớp kết nối đầy đủ liên tiếp, mỗi lớp được theo sau bởi hàm kích hoạt ReLU nhằm tăng khả năng biểu diễn phi tuyến của mô hình.

Kiến trúc của nhánh xử lý vector trạng thái được xây dựng như sau:

- Lớp kết nối đầy đủ thứ nhất: chuyển đổi vector trạng thái từ 21 chiều sang 128 chiều.
- Hàm kích hoạt ReLU.
- Lớp kết nối đầy đủ thứ hai: chuyển đổi vector đặc trưng từ 128 chiều xuống 64 chiều.
- Hàm kích hoạt ReLU.

Quá trình biến đổi có thể được biểu diễn như sau:

$$21 \rightarrow 128 \rightarrow 64$$

Sau quá trình xử lý, vector trạng thái ban đầu được chuyển đổi thành một vector đặc trưng có kích thước 64 phần tử. Vector này chứa các đặc trưng đã được trích xuất và biểu diễn ở mức trừu tượng cao hơn, giúp mô hình học được các mối quan hệ giữa trạng thái UAV, khoảng cách tới vật cản và xu hướng di chuyển tới mục tiêu.

Các đặc trưng thu được từ nhánh Fully Connected sau đó sẽ được kết hợp với đặc trưng ảnh từ nhánh CNN để tạo thành một biểu diễn trạng thái thống nhất, làm đầu vào cho kiến trúc Dueling Network trong mô hình D3QN.

3.3.2.3 Hợp nhất đặc trưng

Sau khi hoàn thành quá trình trích xuất đặc trưng từ ảnh quan sát và vector trạng thái, hai nguồn thông tin này được kết hợp nhằm tạo ra một biểu diễn trạng thái thống nhất cho UAV. Việc sử dụng đồng thời dữ liệu hình ảnh và dữ liệu trạng thái cho phép tác tử khai thác cả thông tin về môi trường bên ngoài lẫn trạng thái nội tại của thiết bị bay, từ đó nâng cao chất lượng ra quyết định trong quá trình điều hướng.

Đối với nhánh xử lý ảnh, sau khi đi qua các tầng tích chập và hàm kích hoạt ReLU, đặc trưng thu được được làm phẳng (*Flatten*) thành một vector có kích thước 3072 phần tử. Trong khi đó, vector trạng thái 21 chiều được đưa qua các tầng Fully Connected để tạo thành một vector đặc trưng có kích thước 64 phần tử.

Hai vector đặc trưng này được ghép nối theo chiều đặc trưng (*Feature Concatenation*) để tạo thành vector đầu vào cho các tầng quyết định phía sau:

$$F_{combined} = [F_{image}, F_{state}]$$

Trong đó:

- F_{image} : vector đặc trưng được trích xuất từ ảnh quan sát.
- F_{state} : vector đặc trưng được trích xuất từ vector trạng thái của UAV.
- $F_{combined}$: vector đặc trưng tổng hợp sau khi ghép nối.

Kích thước của vector tổng hợp được xác định như sau:

$$|F_{combined}| = |F_{image}| + |F_{state}| = 3072 + 64 = 3136$$

Vector đặc trưng tổng hợp này chứa đồng thời thông tin về môi trường quan sát được thông qua ảnh đầu vào và thông tin động học của UAV như vị trí, vận tốc, góc quay, dữ liệu cảm biến khoảng cách và Action Hints. Nhờ đó, tác tử có thể xây dựng một biểu diễn trạng thái toàn diện hơn so với việc chỉ sử dụng một nguồn dữ liệu riêng lẻ.

Vector $F_{combined}$ sau đó được đưa vào kiến trúc Dueling Network, nơi mô hình tách thành hai nhánh độc lập để học giá trị trạng thái $V(s)$ và lợi thế hành động $A(s, a)$ trước khi kết hợp thành hàm giá trị hành động $Q(s, a)$. Đây là đầu vào trực tiếp cho quá trình ra quyết định của tác tử D3QN.

3.3.2.4 Kiến trúc Dueling

Sau khi thu được vector đặc trưng tổng hợp từ quá trình hợp nhất dữ liệu, mô hình tiếp tục sử dụng kiến trúc Dueling Network để ước lượng giá trị hành động. Khác với DQN truyền thống chỉ dự đoán trực tiếp giá trị ($Q(s,a)$), kiến trúc Dueling tách quá trình đánh giá thành hai luồng riêng biệt gồm luồng giá trị trạng thái (Value Stream) và luồng lợi thế hành động (Advantage Stream).

Luồng giá trị trạng thái có nhiệm vụ đánh giá mức độ tốt của trạng thái hiện tại mà không phụ thuộc vào hành động cụ thể $V(s)$. Trong khi đó, luồng lợi thế hành động đánh giá mức độ hiệu quả của từng hành động so với các hành động còn lại tại cùng một trạng thái $A(s,a)$.

Sau khi tính toán hai thành phần trên, giá trị (Q) cuối cùng được xác định theo công thức:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a'; \theta, \alpha) \right)$$

Trong đó:

- $Q(s, a; \theta, \alpha, \beta)$: giá trị Q (Q-value), biểu diễn tổng phần thưởng kỳ vọng mà tác tử có thể nhận được khi thực hiện hành động a tại trạng thái s .
- $V(s; \theta, \beta)$: giá trị trạng thái (State Value), phản ánh mức độ tốt của trạng thái s mà không xét đến hành động cụ thể được thực hiện.
- $A(s, a; \theta, \alpha)$: giá trị lợi thế (Advantage Value), biểu diễn mức độ tốt hơn hoặc kém hơn của hành động a so với các hành động khác tại cùng trạng thái s .
- $\mathcal{A}$: tập hợp tất cả các hành động mà tác tử có thể thực hiện.
- θ : tập tham số của phần trích xuất đặc trưng chung .
- α : tập tham số của nhánh Advantage Stream.
- β : tập tham số của nhánh Value Stream.

Việc trừ đi giá trị trung bình của luồng Advantage giúp mô hình tránh tình trạng không xác định giữa hai thành phần ($V(s)$) và ($A(s,a)$), đồng thời cải thiện tính ổn định trong quá trình huấn luyện.

Trong đồ án này, sau khi nhận vector đặc trưng tổng hợp có kích thước 3136 phần tử, mô hình chia thành hai nhánh xử lý độc lập. Nhánh Value Stream tạo ra một giá trị duy nhất biểu diễn chất lượng của trạng thái hiện tại, trong khi nhánh Advantage Stream tạo ra một giá trị tương ứng với mỗi hành động có thể thực hiện. Hai kết quả này sau đó được kết hợp để tạo thành các giá trị (Q)-value cuối cùng phục vụ quá trình lựa chọn hành động. lựa chọn hành động.

3.3.2.5 Cơ chế học Double DQN

Mặc dù DQN đã đạt được nhiều thành công trong các bài toán học tăng cường, thuật toán này vẫn tồn tại một hạn chế quan trọng là hiện tượng đánh giá quá cao giá trị hành động (Overestimation Bias). Nguyên nhân xuất phát từ việc cùng một mạng nơ-ron vừa được sử dụng để lựa chọn hành động tối ưu vừa được sử dụng để đánh giá giá trị của hành động đó. Sai số xuất hiện trong quá trình dự đoán có thể khiến các giá trị (Q)-value bị thổi phồng, làm giảm tính ổn định và tốc độ hội tụ của mô hình.

Để khắc phục hạn chế này, đề án sử dụng cơ chế Double DQN. Ý tưởng chính của Double DQN là tách riêng quá trình lựa chọn hành động và quá trình đánh giá hành động bằng cách sử dụng hai mạng nơ-ron khác nhau gồm mạng chính (Online Network) và mạng mục tiêu (Target Network).

Trong trạng thái kế tiếp (s'), mạng chính được sử dụng để lựa chọn hành động có giá trị lớn nhất:

$$a^* = \arg \max_{a'} Q(s', a'; \theta)$$

Trong đó:

- a^* : hành động tối ưu được lựa chọn tại trạng thái kế tiếp.
- $Q(s', a'; \theta)$: giá trị Q được dự đoán bởi mạng chính.
- θ : tham số của mạng chính.

Sau khi xác định được hành động tối ưu, mạng mục tiêu sẽ được sử dụng để đánh giá giá trị của hành động đó:

$$Y^{DDQN} = R_{t+1} + \gamma Q\left(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-\right)$$

Trong đó:

- Y^{DDQN} : giá trị mục tiêu dùng để huấn luyện mạng.
- R_{t+1} : phần thưởng nhận được sau khi thực hiện hành động.
- γ : hệ số chiết khấu.
- θ : tham số của mạng chính.
- θ^- : tham số của mạng mục tiêu.

Việc tách riêng quá trình lựa chọn và đánh giá hành động giúp giảm đáng kể hiện tượng overestimation bias, từ đó nâng cao tính ổn định của quá trình huấn luyện và cải thiện chất lượng chính sách học được.

3.3.2.6 Cập nhật mạng mục tiêu

Trong cơ chế Double DQN, bên cạnh mạng chính (Online Network), một mạng mục tiêu (Target Network) được duy trì nhằm cung cấp các giá trị mục tiêu ổn định trong quá trình huấn luyện. Nếu sử dụng trực tiếp mạng chính để vừa dự đoán giá trị hiện tại vừa tạo giá trị mục tiêu, các giá trị (Q)-value sẽ liên tục thay đổi sau mỗi lần cập nhật trọng số, dẫn đến hiện tượng dao động và khó hội tụ.

Để giải quyết vấn đề này, Double DQN sử dụng một mạng mục tiêu có cấu trúc giống hệt mạng chính nhưng được cập nhật chậm hơn. Ban đầu, toàn bộ trọng số của mạng mục tiêu được sao chép từ mạng chính:

$$\theta^- = \theta$$

Trong đó:

- θ : tham số của mạng chính.
- θ^- : tham số của mạng mục tiêu.

Thông thường, trong các nghiên cứu về học tăng cường, mạng mục tiêu được cập nhật định kỳ bằng cách sao chép toàn bộ trọng số từ mạng chính sau một số bước huấn luyện nhất định. Tuy nhiên, trong đề án này, phương pháp cập nhật mềm (Soft Update) được sử dụng thay cho cơ chế cập nhật cứng truyền thống.

Ý tưởng của Soft Update là cập nhật trọng số của mạng mục tiêu một cách từ từ thông qua việc kết hợp giữa trọng số hiện tại của mạng mục tiêu và trọng số mới từ mạng chính. Nhờ đó, sự thay đổi của mạng mục tiêu diễn ra mượt mà hơn, tránh những biến động đột ngột trong quá trình học.

Phương pháp này giúp tăng tính ổn định của quá trình huấn luyện, giảm nguy cơ mất hội tụ do các cập nhật quá lớn, đồng thời cho phép tác tử tiếp thu kiến thức mới mà vẫn duy trì được những kinh nghiệm đã học trước đó. Kết quả là tác tử có khả năng học tập hiệu quả hơn và đạt được chính sách điều khiển ổn định hơn trong môi trường huấn luyện.

$$\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$$

- θ : trọng số của mạng chính (Main Network).
- θ^- : trọng số của mạng mục tiêu (Target Network).
- τ : hệ số cập nhật mềm ($0 < \tau < 1$).

3.3.2.7 Hàm mất mát và tối ưu hóa

Sau khi xác định được giá trị mục tiêu bằng cơ chế Double DQN, tác tử cần điều chỉnh các trọng số của mạng nơ-ron sao cho giá trị (Q)-value dự đoán tiến gần hơn tới giá trị mục tiêu. Quá trình này được thực hiện thông qua hàm mất mát (Loss Function).

Trong đề án này, hàm mất mát được sử dụng là Smooth L1 Loss (Huber Loss). Hàm mất mát này được tính dựa trên sai lệch giữa giá trị (Q)-value hiện tại và giá trị mục tiêu:

$$L = \text{SmoothL1} (Q(s, a), Y^{DDQN})$$

Trong đó:

- L : giá trị hàm mất mát.
- $Q(s, a)$: giá trị Q hiện tại được dự đoán bởi mạng chính.
- Y^{DDQN} : giá trị mục tiêu được tính bằng cơ chế Double DQN.

So với Mean Squared Error (MSE), Huber Loss có khả năng giảm ảnh hưởng của các sai số quá lớn xuất hiện trong giai đoạn đầu huấn luyện. Điều này giúp quá trình học ổn định hơn và hạn chế hiện tượng dao động mạnh của gradient.

Sau khi tính được hàm mất mát, mô hình sử dụng thuật toán tối ưu Adam để cập nhật các trọng số của mạng nơ-ron. Adam là một trong những thuật toán tối ưu phổ biến trong học sâu nhờ khả năng tự động điều chỉnh tốc độ học cho từng tham số, từ đó giúp mô hình hội tụ nhanh và ổn định hơn.

Bên cạnh đó, đề án còn áp dụng kỹ thuật Gradient Clipping nhằm giới hạn giá trị gradient trong khoảng:

$$-1 \leq g \leq 1$$

Trong đó:

- g : giá trị gradient của một tham số bất kỳ trong mạng nơ-ron.

Kỹ thuật này giúp ngăn chặn hiện tượng gradient bùng nổ (Exploding Gradient), đặc biệt trong giai đoạn đầu huấn luyện khi sai số giữa giá trị dự đoán và giá trị mục tiêu còn lớn.

Thông qua sự kết hợp giữa hàm mất mát Huber Loss, bộ tối ưu Adam và kỹ thuật Gradient Clipping, quá trình huấn luyện của tác tử trở nên ổn định hơn, giảm dao động trong quá trình cập nhật trọng số và cải thiện khả năng hội tụ của mô hình.

3.3.3 Các siêu tham số huấn luyện

Bên cạnh kiến trúc mạng nơ-ron và thuật toán học tăng cường, việc lựa chọn các siêu tham số huấn luyện (Hyperparameters) đóng vai trò quan trọng đối với hiệu quả hội tụ của mô hình. Các siêu tham số này ảnh hưởng trực tiếp đến tốc độ học, khả năng khám phá môi trường cũng như độ ổn định của quá trình huấn luyện.

Trong đề án này, các siêu tham số được lựa chọn dựa trên các nghiên cứu trước đây về DQN, DDQN và Dueling DQN, đồng thời được điều chỉnh thông qua quá trình thực nghiệm để phù hợp với bài toán điều hướng UAV trong môi trường có vật cản.

Bảng 3.1. Các siêu tham số huấn luyện

Siêu tham số	Giá trị
Learning Rate	$4 * 10^{-5}$
Batch Size	64
Replay Buffer Size	50000
Discount Factor (γ)	0.95
Initial Epsilon	1.0
Minimum Epsilon	0.1
Epsilon Decay Period	200 episode
Soft Update Parameter	0.005
Optimizer	Adam
Loss Function	Smooth L1 Loss

Chương 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 So sánh với mô hình DQN truyền thống

Nhằm xây dựng một mô hình cơ sở để đánh giá hiệu quả của thuật toán D3QN, đồ án tiến hành so sánh với thuật toán DQN truyền thống [6]. Các siêu tham số được thể hiện ở bảng sau:

Bảng 4.1. Các siêu tham số huấn luyện của tác tử sử dụng DQN truyền thống

Siêu tham số	Giá trị
Learning Rate	$4 * 10^{-5}$
Batch Size	64
Replay Buffer Size	50000
Discount Factor (γ)	0.95
Initial Epsilon	1.0
Minimum Epsilon	0.1
Epsilon Decay Period	200 episode
Soft Update Parameter	0.005
Optimizer	Adam
Loss Function	Smooth L1 Loss
Gradient Clipping	[-1, 1]

Tác tử sử dụng DQN truyền thống có các siêu tham số tương tự với D3QN được nhắc đến ở bảng 3.1. Tuy nhiên ở đây sự khác biệt nằm ở kiến trúc mạng ra quyết định.

```
with torch.no_grad():
    next_q_values = self.target_model(
        ns_img_t, ns_vec_t).max(1)[0]
    target_q_values = r_t + (
        self.gamma * next_q_values * (1 - d_t))
```

Còn ở thuật toán D3QN sử dụng cả kiến trúc DDQN và Dueling Network Architecture:

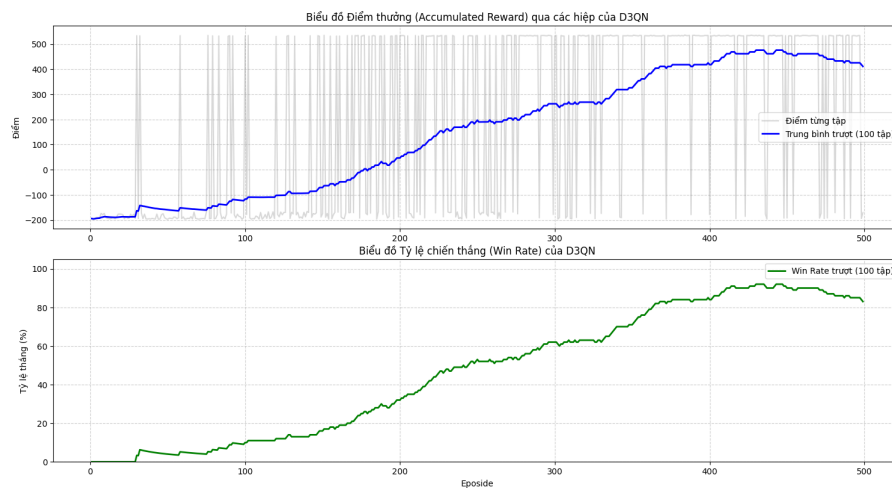
```
values = self.value_stream(combined)
advantages = self.advantage_stream(combined)
```

4.1. SO SÁNH VỚI MÔ HÌNH DQN TRUYỀN THỐNG

```
q_values = values + (  
    advantages -  
    advantages.mean(dim=1, keepdim=True)  
)
```

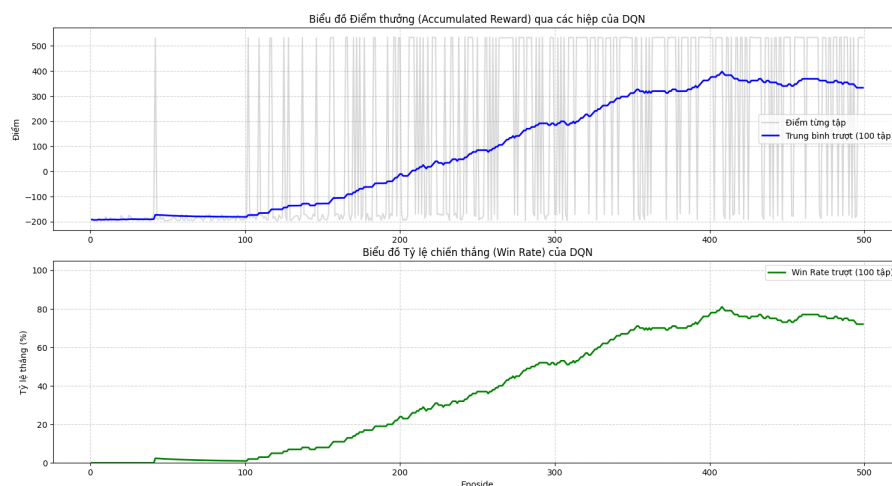
```
next_q_values =  
self.target_model(ns_img_t, ns_vec_t)  
.gather(1, next_actions)
```

Kết quả huấn luyện của hai mô hình được thể hiện trong Hình 4.1 và Hình 4.2. Có thể nhận thấy cả DQN và D3QN đều có khả năng học được chính sách điều hướng UAV khi phần thưởng tích lũy và tỷ lệ chiến thắng đều tăng theo thời gian huấn luyện.



Hình 4.1. Kết quả huấn luyện của tác tử D3QN

4.1. SO SÁNH VỚI MÔ HÌNH DQN TRUYỀN THỐNG



Hình 4.2. Kết quả huấn luyện của tác tử DQN

Hình 4.1 và Hình 4.2 lần lượt thể hiện sự thay đổi của phần thưởng tích lũy (Accumulated Reward) và tỷ lệ chiến thắng (Win Rate) trong quá trình huấn luyện của hai mô hình DQN và D3QN. Các đường màu xanh biểu diễn giá trị trung bình trượt trên 100 episode nhằm làm giảm ảnh hưởng của nhiễu và phản ánh xu hướng học tập tổng thể của tác tử.

Quan sát biểu đồ phần thưởng tích lũy cho thấy cả hai mô hình đều có khả năng học hỏi từ môi trường khi giá trị phần thưởng trung bình tăng dần theo số episode huấn luyện. Trong giai đoạn đầu (khoảng từ episode 0 đến episode 150), cả DQN và D3QN đều đạt mức phần thưởng thấp, dao động quanh giá trị âm do tác tử chủ yếu thực hiện các hành động mang tính khám phá và chưa hình thành được chiến lược điều hướng hiệu quả. Tuy nhiên, từ khoảng episode 150 trở đi, sự khác biệt giữa hai mô hình bắt đầu xuất hiện rõ rệt.

Đối với mô hình DQN truyền thống, phần thưởng tích lũy tăng tương đối chậm và chỉ đạt giá trị cực đại khoảng 390–400 điểm ở giai đoạn cuối huấn luyện. Sau khi đạt đỉnh, đường phần thưởng có xu hướng dao động và giảm nhẹ, cho thấy chính sách học được vẫn chưa thực sự ổn định. Trong khi đó, mô hình D3QN thể hiện tốc độ cải thiện nhanh hơn đáng kể. Từ khoảng episode 200, giá trị phần thưởng tăng mạnh và liên tục, đạt đỉnh khoảng 470–480 điểm. Ngay cả ở giai đoạn cuối huấn luyện, phần thưởng trung bình của D3QN vẫn duy trì ở mức trên 400 điểm, cao hơn đáng kể so với DQN.

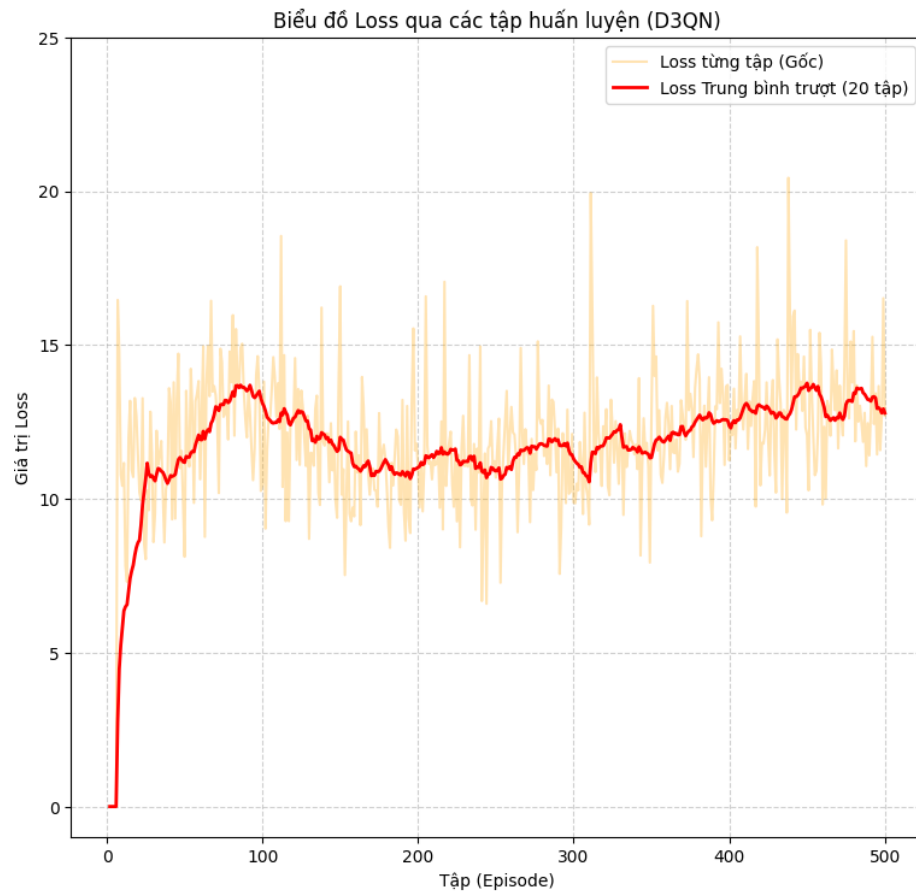
Sự khác biệt này tiếp tục được thể hiện rõ thông qua biểu đồ tỷ lệ chiến thắng.

Trong giai đoạn đầu huấn luyện, tỷ lệ chiến thắng của cả hai mô hình gần như bằng 0 do tác tử chưa học được cách di chuyển tới đích và tránh vật cản hiệu quả. Khi số lượng episode tăng lên, cả hai mô hình đều cải thiện khả năng hoàn thành nhiệm vụ, tuy nhiên D3QN đạt tốc độ tăng trưởng nhanh hơn. Tại khoảng episode 300, tỷ lệ chiến thắng của D3QN đã vượt ngưỡng 60%, trong khi DQN vẫn chỉ dao động quanh mức 50%.

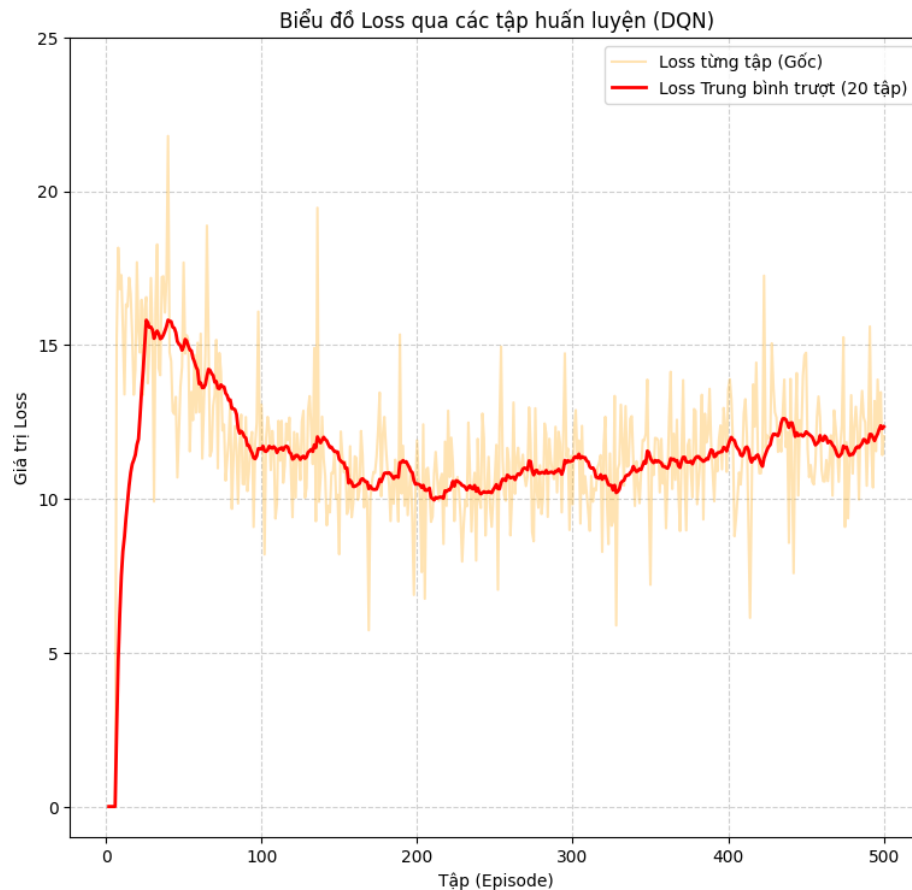
Ở giai đoạn cuối quá trình huấn luyện, D3QN đạt tỷ lệ chiến thắng cao nhất xấp xỉ 90% và duy trì ổn định trong khoảng 85–90%. Ngược lại, DQN chỉ đạt mức cao nhất khoảng 80% và phần lớn thời gian duy trì trong khoảng 72–78%. Điều này cho thấy D3QN không chỉ học được chính sách điều hướng hiệu quả hơn mà còn có khả năng duy trì hiệu suất ổn định trong thời gian dài.

Nguyên nhân của sự cải thiện này đến từ việc D3QN kết hợp đồng thời hai cơ chế quan trọng là Double Deep Q-Learning (DDQN) và Dueling Network Architecture. Trong DQN truyền thống, cùng một mạng được sử dụng để vừa lựa chọn hành động tối ưu vừa đánh giá giá trị của hành động đó. Điều này dễ dẫn đến hiện tượng đánh giá quá cao giá trị Q (Overestimation Bias), khiến quá trình học trở nên thiếu ổn định. DDQN khắc phục hạn chế này bằng cách tách riêng quá trình lựa chọn hành động và quá trình đánh giá hành động thông qua hai mạng khác nhau, giúp giảm sai lệch trong quá trình cập nhật giá trị mục tiêu.

Bên cạnh đó, kiến trúc Dueling Network Architecture cho phép mạng nơ-ron học riêng biệt hai thành phần là giá trị trạng thái ($V(s)$) và lợi thế hành động ($A(s,a)$). Nhờ vậy, mô hình có thể đánh giá mức độ "tốt" của trạng thái hiện tại ngay cả khi sự khác biệt giữa các hành động là không đáng kể. Điều này giúp quá trình học diễn ra hiệu quả hơn, đặc biệt trong các bài toán điều hướng UAV khi nhiều hành động có thể dẫn đến kết quả tương tự nhau.



Hình 4.3. Biểu đồ loss của D3QN



Hình 4.4. Biểu đồ loss của DQN

Kết quả ở Hình 4.3 và Hình 4.4 cho thấy giá trị Loss của cả DQN và D3QN đều giảm đáng kể trong giai đoạn đầu huấn luyện trước khi ổn định ở các episode sau. Điều này chứng tỏ cả hai mô hình đều học được mối quan hệ giữa trạng thái, hành động và phần thưởng thông qua quá trình tương tác với môi trường.

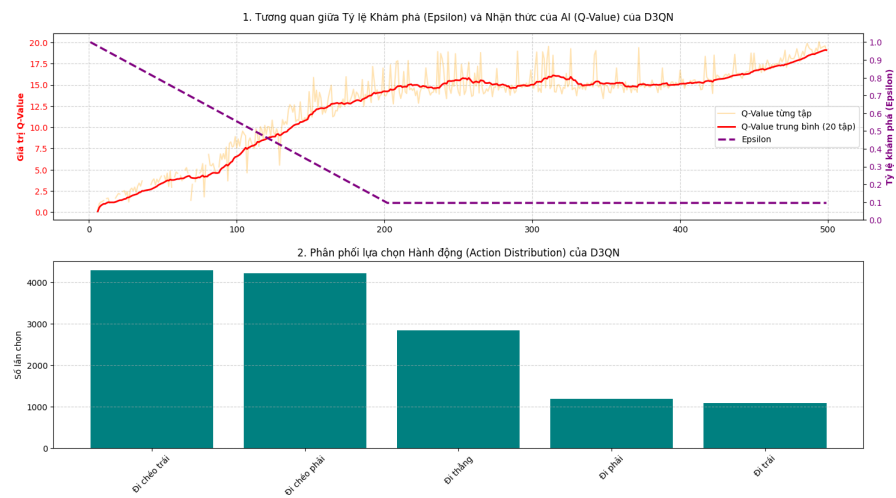
Ở giai đoạn đầu, D3QN có mức Loss thấp hơn so với DQN, cho thấy quá trình cập nhật giá trị Q diễn ra ổn định hơn. Điều này có thể được giải thích bởi cơ chế Double Deep Q-Learning giúp giảm hiện tượng đánh giá quá cao giá trị hành động, từ đó làm giảm sai số trong quá trình cập nhật mạng nơ-ron.

Trong giai đoạn hội tụ, giá trị Loss của hai mô hình dao động trong khoảng từ 10 đến 13 và không có sự khác biệt quá lớn. Tuy nhiên, khi đối chiếu với các kết quả về Accumulated Reward và Win Rate, có thể nhận thấy D3QN đạt hiệu năng tốt hơn đáng kể mặc dù giá trị Loss không thấp hơn DQN. Điều này cho thấy Loss không phải là chỉ số duy nhất phản ánh chất lượng của tác tử trong bài toán học tăng cường.

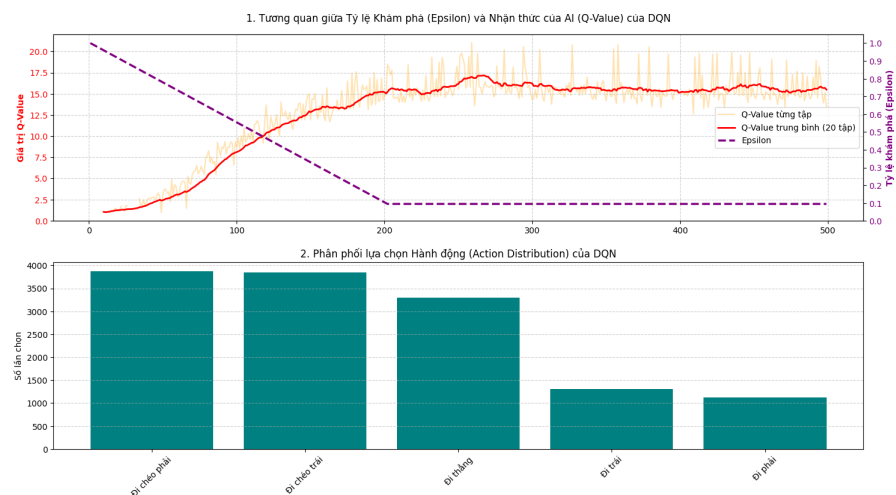
4.1. SO SÁNH VỚI MÔ HÌNH DQN TRUYỀN THỐNG

Nguyên nhân là do D3QN sử dụng kiến trúc Dueling Network Architecture để học đồng thời giá trị trạng thái và lợi thế hành động, giúp mô hình khai thác thông tin trạng thái hiệu quả hơn. Kết quả là tác tử có thể đưa ra các quyết định chính xác hơn, đạt phần thưởng tích lũy cao hơn và tỷ lệ chiến thắng lớn hơn so với DQN truyền thống.

Từ các kết quả thực nghiệm có thể kết luận rằng mặc dù mức Loss của D3QN và DQN là tương đương, D3QN vẫn thể hiện khả năng học chính sách điều hướng vượt trội hơn, chứng minh hiệu quả của việc kết hợp cơ chế DDQN và Dueling Network trong bài toán điều hướng UAV tự động.



Hình 4.5. Biểu đồ phân phối hành động và giá trị Q-Values của D3QN



Hình 4.6. Biểu đồ phân phối hành động và giá trị Q-Values của DQN

Kết quả trong Hình 4.5 và Hình 4.6 thể hiện mối quan hệ giữa hệ số khám phá ϵ , giá trị Q trung bình (Q-Value) và phân bố hành động của hai mô hình DQN và

D3QN trong suốt quá trình huấn luyện. Các biểu đồ này cho phép đánh giá sâu hơn về khả năng học tập, mức độ hội tụ cũng như hành vi ra quyết định của tác tử trong môi trường điều hướng UAV tránh vật cản.

Quan sát đồ thị Q-Value cho thấy cả hai mô hình đều có xu hướng gia tăng giá trị Q khi hệ số khám phá ϵ giảm dần từ 1.0 xuống 0.1. Trong giai đoạn đầu huấn luyện, tác tử chủ yếu thực hiện các hành động ngẫu nhiên nhằm thu thập dữ liệu về môi trường. Do đó, giá trị Q còn ở mức thấp và tăng tương đối chậm. Khi ϵ giảm xuống, tác tử bắt đầu khai thác các kinh nghiệm đã học được từ bộ nhớ Experience Replay, giúp các quyết định trở nên hiệu quả hơn và làm cho giá trị Q tăng lên đáng kể.

Đối với mô hình DQN, giá trị Q trung bình tăng nhanh từ khoảng episode 50 đến episode 250, phản ánh việc tác tử đã học được các chiến lược điều hướng cơ bản để tiến tới mục tiêu và tránh va chạm với vật cản. Tuy nhiên, sau giai đoạn này, tốc độ tăng trưởng của Q-Value giảm dần và đường cong xuất hiện nhiều dao động nhỏ quanh mức từ 15 đến 17 điểm. Điều này cho thấy mô hình bắt đầu tiến gần đến trạng thái hội tụ nhưng vẫn tồn tại sự không ổn định trong quá trình đánh giá giá trị hành động. Nguyên nhân chủ yếu đến từ hiện tượng overestimation bias của DQN, khi cùng một mạng nơ-ron vừa thực hiện lựa chọn hành động tối ưu vừa đánh giá giá trị của hành động đó.

Ngược lại, mô hình D3QN cho thấy quá trình học ổn định hơn. Giá trị Q trung bình tăng tương đối đều trong toàn bộ quá trình huấn luyện và tiếp tục cải thiện ở các episode cuối cùng. Đến episode 500, giá trị Q trung bình của D3QN đạt xấp xỉ 19 điểm, cao hơn đáng kể so với DQN. Ngoài ra, đường cong Q-Value của D3QN có mức dao động thấp hơn và ít xuất hiện các biến động đột ngột. Điều này cho thấy tác tử có khả năng đánh giá phần thưởng tương lai chính xác hơn, từ đó đưa ra các quyết định ổn định hơn trong môi trường huấn luyện.

Sự khác biệt này có thể được giải thích bởi hai thành phần chính của D3QN. Thứ nhất, cơ chế Double Deep Q-Learning giúp tách biệt quá trình lựa chọn hành động và đánh giá hành động, từ đó làm giảm hiện tượng đánh giá quá mức giá trị Q. Thứ hai, kiến trúc Dueling Network phân tách việc học giá trị trạng thái $V(s)$ và lợi thế hành động $A(s, a)$ thành hai nhánh riêng biệt trước khi kết hợp thành hàm $Q(s, a)$. Nhờ đó, mô hình có khả năng nhận biết những trạng thái có lợi ngay cả khi sự khác

biệt giữa các hành động là không lớn, giúp quá trình học diễn ra hiệu quả hơn.

Bên cạnh việc đánh giá Q-Value, biểu đồ phân bố hành động cũng cung cấp cái nhìn trực quan về chính sách mà tác tử đã học được. Kết quả cho thấy cả hai mô hình đều ưu tiên sử dụng các hành động “Đi chéo trái” và “Đi chéo phải” với tần suất cao nhất. Đây là điều hoàn toàn hợp lý bởi trong bài toán điều hướng UAV, các hành động di chuyển theo đường chéo cho phép thiết bị vừa tiến về phía mục tiêu vừa thay đổi hướng để tránh vật cản. So với việc chỉ di chuyển sang trái hoặc sang phải, các hành động chéo mang lại hiệu quả cao hơn trong việc rút ngắn quãng đường di chuyển và giảm số bước cần thiết để hoàn thành nhiệm vụ.

Tuy nhiên, khi so sánh chi tiết có thể nhận thấy sự khác biệt giữa hai mô hình. Ở DQN, tần suất lựa chọn hành động tập trung nhiều hơn vào một số hành động nhất định, cho thấy tác tử có xu hướng phụ thuộc vào các chiến lược quen thuộc đã mang lại phần thưởng cao trong quá khứ. Điều này có thể giúp tăng hiệu quả trong một số tình huống cụ thể nhưng cũng làm giảm khả năng thích nghi khi môi trường thay đổi.

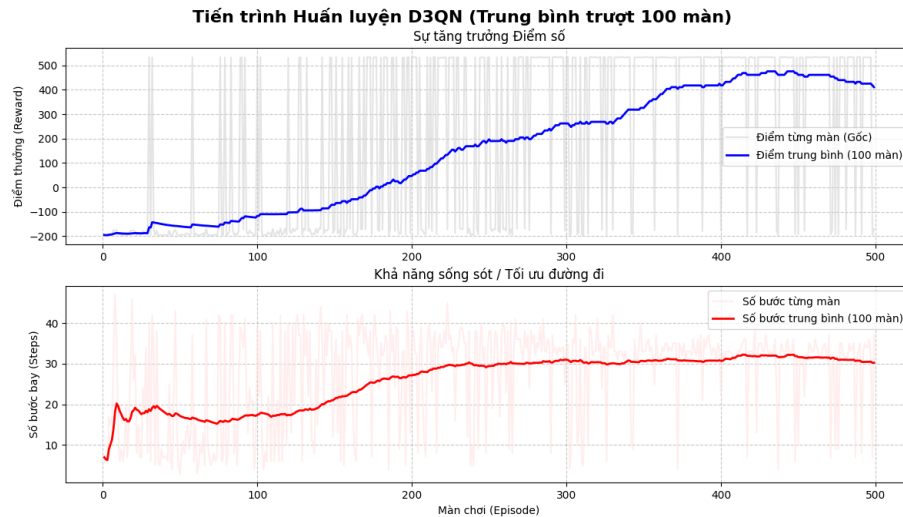
Trong khi đó, mô hình D3QN thể hiện sự phân bố hành động cân bằng hơn. Mặc dù các hành động đi chéo vẫn chiếm ưu thế, nhưng tần suất lựa chọn các hành động khác không bị chênh lệch quá lớn. Điều này cho thấy tác tử đã học được nhiều phương án điều hướng khác nhau thay vì chỉ phụ thuộc vào một số chiến lược cố định. Nhờ khả năng đánh giá trạng thái chính xác hơn, D3QN có thể linh hoạt lựa chọn hành động phù hợp với từng tình huống cụ thể trong môi trường.

Một điểm đáng chú ý khác là sau khi hệ số khám phá giảm xuống mức tối thiểu 0.1, giá trị Q của D3QN vẫn tiếp tục tăng trong khi DQN có xu hướng chững lại. Điều này chứng tỏ D3QN vẫn có khả năng khai thác hiệu quả các kinh nghiệm đã học được để cải thiện chính sách điều khiển, trong khi DQN dần đạt tới giới hạn về khả năng biểu diễn và học tập. Kết quả này phù hợp với các nghiên cứu trước đây khi chỉ ra rằng việc kết hợp Double Q-Learning và Dueling Network Architecture có thể giúp tăng tốc độ hội tụ cũng như nâng cao chất lượng chính sách cuối cùng.

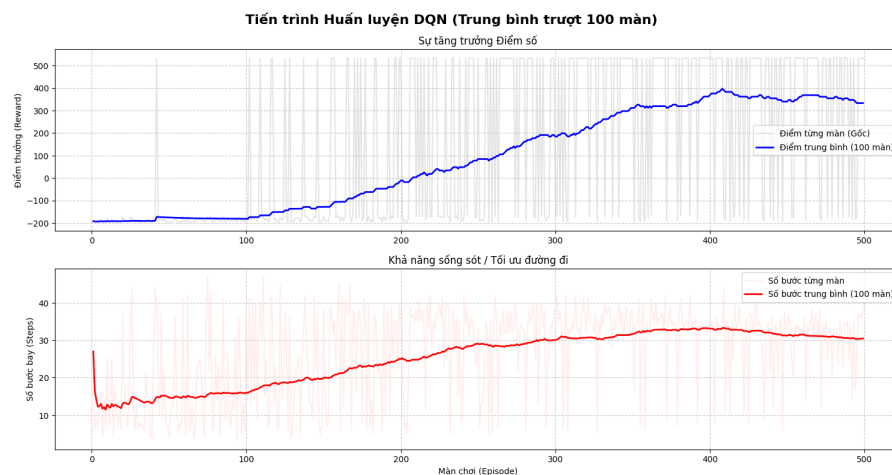
Tổng hợp các kết quả phân tích từ Q-Value, hệ số khám phá và phân bố hành động cho thấy mô hình D3QN không chỉ học được chính sách điều hướng hiệu quả hơn mà còn duy trì được tính ổn định cao trong suốt quá trình huấn luyện. Giá trị

4.1. SO SÁNH VỚI MÔ HÌNH DQN TRUYỀN THỐNG

Q cao hơn, mức dao động thấp hơn và khả năng sử dụng hành động linh hoạt hơn là những bằng chứng cho thấy D3QN đã khai thác môi trường tốt hơn so với DQN truyền thống. Đây cũng là nguyên nhân dẫn đến việc D3QN đạt điểm thưởng tích lũy cao hơn và tỷ lệ chiến thắng tốt hơn trong các thí nghiệm đánh giá đã trình bày ở các phần trước.



Hình 4.7. Biểu đồ điểm tích lũy và số bước trung bình của D3QN



Hình 4.8. Biểu đồ điểm tích lũy và số bước trung bình của DQN

Ở Hình 4.7 và Hình 4.8 trình bày sự thay đổi của điểm thưởng tích lũy (Accumulated Reward) và số bước trung bình (Average Steps) trong quá trình huấn luyện của hai thuật toán D3QN và DQN.

Xét trên phương diện số bước trung bình, tại episode thứ 500, cả hai thuật toán đều đạt giá trị xấp xỉ 30 bước cho mỗi tập huấn luyện. Điều này cho thấy cả DQN

và D3QN đều đã học được chính sách điều hướng đủ tốt để duy trì khả năng sống sót trong môi trường. Tuy nhiên, khi quan sát toàn bộ quá trình huấn luyện có thể nhận thấy sự khác biệt đáng kể giữa hai thuật toán.

Trong giai đoạn đầu từ episode 0 đến khoảng episode 200, số bước trung bình của D3QN thường cao hơn DQN, cho thấy tác tử D3QN thích nghi với môi trường nhanh hơn và có khả năng duy trì trạng thái hoạt động lâu hơn. Đặc biệt, từ sau episode 200, D3QN nhanh chóng đạt ngưỡng ổn định khoảng 30 bước và duy trì giá trị này trong phần lớn thời gian huấn luyện. Điều này phản ánh khả năng hội tụ tốt và tính ổn định cao của thuật toán.

Ngược lại, DQN có tốc độ cải thiện chậm hơn. Mặc dù số bước trung bình vẫn tiếp tục tăng và thậm chí đạt mức cao hơn D3QN tại một số thời điểm, đặc biệt quanh episode 400, nhưng sự biến động của đường cong lớn hơn và thời gian để đạt trạng thái ổn định cũng dài hơn. Điều này cho thấy tác tử DQN vẫn đang trong quá trình điều chỉnh chính sách và chưa đạt được mức hội tụ ổn định như D3QN.

Tuy nhiên, nếu chỉ đánh giá dựa trên số bước sống sót thì chưa thể phản ánh đầy đủ chất lượng của chính sách điều khiển. Khi xem xét biểu đồ điểm thưởng tích lũy, có thể thấy từ khoảng episode 200 trở đi, D3QN liên tục duy trì mức điểm cao hơn DQN. Khoảng cách này ngày càng rõ rệt ở giai đoạn cuối của quá trình huấn luyện.

Kết quả trên cho thấy mặc dù DQN có thể đạt số bước sống sót tương đương hoặc thậm chí cao hơn D3QN tại một số thời điểm, nhưng các hành động được lựa chọn chưa thực sự tối ưu về mặt phần thưởng. Nói cách khác, tác tử DQN có thể duy trì hoạt động trong môi trường nhưng chưa tận dụng được các chiến lược mang lại hiệu quả cao nhất. Trong khi đó, D3QN không chỉ duy trì được khả năng sống sót mà còn học được các hành vi giúp tối đa hóa phần thưởng nhận được trong suốt quá trình thực hiện nhiệm vụ.

Điều này chứng tỏ D3QN đã xây dựng được một chính sách điều hướng hiệu quả hơn, vừa đảm bảo khả năng tránh vật cản và tồn tại lâu trong môi trường, vừa tối ưu hóa mục tiêu phần thưởng. Sự cải thiện này đến từ việc kết hợp cơ chế Double Deep Q-Learning nhằm giảm hiện tượng đánh giá quá mức giá trị hành động cùng với kiến trúc Dueling Network giúp đánh giá chính xác hơn giá trị trạng thái và lợi thế của từng hành động. Nhờ đó, D3QN thể hiện khả năng hội tụ nhanh hơn, ổn

định hơn và đạt hiệu quả tổng thể vượt trội so với DQN truyền thống.

Từ các kết quả thực nghiệm thu được có thể kết luận rằng D3QN vượt trội hơn DQN truyền thống trên cả hai tiêu chí đánh giá là phần thưởng tích lũy, tỷ lệ chiến thắng biểu đồ Q-Values trên Epsilon. Mô hình không chỉ hội tụ nhanh hơn mà còn đạt hiệu năng cao hơn và ổn định hơn trong suốt quá trình huấn luyện. Điều này chứng minh rằng việc kết hợp cơ chế DDQN với kiến trúc Dueling Network là một hướng tiếp cận phù hợp cho bài toán điều hướng UAV tự động trong môi trường có vật cản.

4.2 Các bài kiểm thử khác nhau

Sau quá trình huấn luyện, mô hình D3QN được đánh giá trên một tập môi trường mới nhằm kiểm tra khả năng tổng quát hóa của tác tử đối với các tình huống chưa từng xuất hiện trong quá trình học.

Trong bài kiểm thử này, các vật cản được sinh ngẫu nhiên với tọa độ $x \in (-6, 6)$ và $y \in (-7.49, 8, 36)$. Các vật cản vẫn giữ nguyên đặc tính như trong quá trình huấn luyện, bao gồm hình dạng hình trụ, bán kính đáy bằng 1 đơn vị và được hiển thị với màu xanh. Việc thay đổi vị trí vật cản ở mỗi lần chạy giúp tăng tính đa dạng của môi trường và đánh giá khả năng thích nghi của mô hình.

Mỗi lần kiểm thử bao gồm 500 episode độc lập và quá trình đánh giá được thực hiện qua hai đợt kiểm thử khác nhau. Kết quả thu được được trình bày trong Bảng 4.2 và 4.3 .

Lần	Tổng số episode	Episode chiến thắng	Episode thất bại		
			Đâm vật cản	Quá số bước	Ra ngoài map
1	100	82	5	0	13
2	100	81	8	0	11
3	100	82	10	0	8
4	100	82	7	0	11
5	100	80	5	0	15
Tổng	500	407	35	0	58
Tỉ lệ	—	81.40%	7.00%	0.00%	11.60%

Bảng 4.2. Kết quả đánh giá mô hình theo nguyên nhân kết thúc episode

4.2. CÁC BÀI KIỂM THỬ KHÁC NHAU

Lần	Tổng số episode	Episode chiến thắng	Episode thất bại		
			Đâm vật cản	Quá số bước	Ra ngoài map
1	100	51	10	0	39
2	100	46	21	0	33
3	100	63	7	0	30
4	100	52	9	0	39
5	100	48	13	0	39
Tổng	500	260	60	0	180
Tỉ lệ	–	52.00%	12.00%	0.00%	36.00%

Bảng 4.3. Kết quả đánh giá mô hình DQN theo nguyên nhân kết thúc episode

Mô hình	Win Rate trung bình	Độ lệch chuẩn	Khoảng dao động
D3QN	81.40%	0.89%	80.00% – 82.00%
DQN	52.00%	6.60%	46.00% – 63.00%

Bảng 4.4. Thống kê tỷ lệ chiến thắng qua các lần kiểm thử

Dựa trên các kết quả thực nghiệm thu được, có thể nhận thấy mô hình D3QN vượt trội hơn đáng kể so với DQN trong bài toán điều hướng UAV tránh vật cản. Trong tổng số 500 episode kiểm thử, D3QN đạt 407 episode thành công, tương ứng tỷ lệ chiến thắng 81.40%, trong khi DQN chỉ đạt 260 episode thành công, tương ứng tỷ lệ chiến thắng 52.00%. Như vậy, D3QN đạt nhiều hơn 147 episode thành công và cải thiện tỷ lệ hoàn thành nhiệm vụ thêm 29.40 điểm phần trăm so với DQN.

Xét về các nguyên nhân thất bại, D3QN chỉ ghi nhận 35 trường hợp va chạm với vật cản, chiếm 7.00% tổng số episode, trong khi DQN có tới 60 trường hợp, tương ứng 12.00%. Điều này cho thấy D3QN có khả năng nhận biết và phản ứng với vật cản hiệu quả hơn, từ đó giảm đáng kể nguy cơ va chạm trong quá trình di chuyển.

Sự khác biệt lớn nhất nằm ở số lượng episode thất bại do UAV di chuyển ra ngoài phạm vi bản đồ. D3QN chỉ ghi nhận 58 trường hợp, tương ứng 11.60%, trong khi DQN có tới 180 trường hợp, tương ứng 36.00%. Nói cách khác, số lần thất bại do ra ngoài bản đồ của DQN cao gấp hơn ba lần so với D3QN. Kết quả này cho thấy D3QN học được chính sách điều hướng ổn định hơn, giúp UAV duy trì quỹ đạo bay hợp lý và kiểm soát hướng di chuyển tốt hơn trong môi trường có nhiều vật cản được sinh ngẫu nhiên.

Bên cạnh hiệu năng trung bình, tính ổn định của mô hình cũng được thể hiện rõ qua kết quả thống kê giữa các lần kiểm thử. D3QN đạt tỷ lệ chiến thắng dao động từ

80.00% đến 82.00%, với độ lệch chuẩn chỉ 0.89%. Trong khi đó, DQN dao động từ 46.00% đến 63.00%, với độ lệch chuẩn lên tới 6.60%. Điều này chứng tỏ hiệu năng của D3QN gần như không thay đổi khi môi trường thay đổi, trong khi DQN bị ảnh hưởng đáng kể bởi vị trí và cách bố trí vật cản.

Nguyên nhân của sự cải thiện này đến từ việc D3QN kết hợp đồng thời hai cơ chế quan trọng là Double Deep Q-Learning và Dueling Network Architecture. Double Deep Q-Learning giúp giảm hiện tượng đánh giá quá mức giá trị hành động, từ đó làm cho quá trình học ổn định hơn và các giá trị Q được ước lượng chính xác hơn. Trong khi đó, Dueling Network Architecture cho phép mô hình học riêng biệt giá trị của trạng thái và lợi thế của từng hành động, giúp tác tử đánh giá chính xác hơn các lựa chọn điều khiển trong những tình huống phức tạp.

Từ các kết quả thực nghiệm trên có thể kết luận rằng D3QN không chỉ cải thiện đáng kể tỷ lệ hoàn thành nhiệm vụ mà còn nâng cao khả năng tránh vật cản, giảm số lần mất kiểm soát quỹ đạo và duy trì hiệu năng ổn định trong các môi trường chưa từng xuất hiện trong quá trình huấn luyện. Điều này chứng minh tính hiệu quả và tính phù hợp của thuật toán D3QN đối với bài toán điều hướng UAV tự động trong môi trường có vật cản ngẫu nhiên. Từ các kết quả thực nghiệm thu được có thể thấy rằng D3QN không chỉ cải thiện tỷ lệ hoàn thành nhiệm vụ mà còn làm giảm đáng kể các trường hợp thất bại do va chạm và mất kiểm soát quỹ đạo. Điều này chứng minh rằng mô hình đã học được chiến lược điều hướng hiệu quả và có khả năng thích nghi tốt với các môi trường chưa từng xuất hiện trong quá trình huấn luyện. Kết quả này cũng khẳng định tính phù hợp của thuật toán D3QN đối với bài toán điều hướng UAV tự động trong môi trường có vật cản ngẫu nhiên.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đồ án đã nghiên cứu và xây dựng thành công một hệ thống điều hướng UAV tự động trong môi trường có vật cản dựa trên phương pháp học tăng cường sâu. Môi trường mô phỏng được xây dựng trên nền tảng PyBullet, cho phép UAV tương tác với các vật cản và học cách di chuyển đến vị trí mục tiêu thông qua quá trình thử và sai.

Trong quá trình thực hiện, đồ án đã xây dựng được các thành phần quan trọng của hệ thống bao gồm môi trường huấn luyện, cơ chế sinh vị trí xuất phát và đích đến, hàm phần thưởng, cơ chế reset cũng như hệ thống quan sát dựa trên ảnh từ camera gắn trên UAV. Ngoài ra, tác tử học tăng cường được thiết kế dựa trên kiến trúc Deep Reinforcement Learning kết hợp giữa Dueling Network và Double DQN nhằm nâng cao chất lượng ước lượng giá trị hành động và cải thiện độ ổn định trong quá trình huấn luyện.

Kết quả thực nghiệm cho thấy các tác tử đều có khả năng học được chính sách điều hướng đến mục tiêu. Trong đó, mô hình D3QN đạt hiệu quả tốt hơn so với DQN về tốc độ hội tụ, độ ổn định cũng như khả năng tránh vật cản trong môi trường mô phỏng. Điều này cho thấy việc kết hợp giữa cơ chế Dueling Network và Double DQN mang lại hiệu quả tích cực đối với bài toán điều hướng UAV.

Thông qua đồ án, tác giả đã có cơ hội tìm hiểu sâu hơn về học tăng cường sâu, các kỹ thuật xây dựng môi trường mô phỏng, thiết kế hàm phần thưởng và huấn luyện tác tử trong các bài toán điều hướng tự động. Những kiến thức và kinh nghiệm thu được là nền tảng quan trọng cho các nghiên cứu và ứng dụng trong tương lai.

5.2 Hướng phát triển

Mặc dù đã đạt được những kết quả nhất định, đề tài vẫn còn một số hạn chế và có thể được mở rộng theo nhiều hướng trong tương lai.

Thứ nhất, môi trường mô phỏng hiện tại vẫn còn tương đối đơn giản với số lượng vật cản và kịch bản huấn luyện còn hạn chế. Trong tương lai, có thể xây dựng các môi trường phức tạp hơn với vật cản động, nhiều loại địa hình hoặc các điều kiện

môi trường khác nhau nhằm nâng cao khả năng thích nghi của tác tử.

Thứ hai, hệ thống hiện tại chủ yếu sử dụng dữ liệu ảnh từ camera và vector trạng thái của UAV. Các nghiên cứu tiếp theo có thể tích hợp thêm các loại cảm biến khác như LiDAR, cảm biến độ sâu hoặc cảm biến khoảng cách để giúp UAV nhận thức môi trường tốt hơn và đưa ra quyết định chính xác hơn.

Thứ ba, đề tài mới chỉ đánh giá trên môi trường mô phỏng. Một hướng nghiên cứu quan trọng là triển khai mô hình trên UAV thực tế hoặc các nền tảng mô phỏng có độ chân thực cao hơn nhằm đánh giá khả năng chuyển giao từ mô phỏng sang thực tế (Sim-to-Real).

Cuối cùng, có thể nghiên cứu và áp dụng các thuật toán học tăng cường hiện đại hơn như PPO, SAC hoặc TD3 để so sánh với D3QN, từ đó tìm ra phương pháp phù hợp nhất cho bài toán điều hướng UAV trong môi trường có vật cản.

TÀI LIỆU THAM KHẢO

- [1] Richard Bellman. *Dynamic Programming*. Princeton, NJ, USA: Princeton University Press, 1957.
- [2] Ronald A. Howard. *Dynamic Programming and Markov Processes*. Cambridge, MA, USA: MIT Press, 1960.
- [3] Richard S. Sutton. “Learning to Predict by the Methods of Temporal Differences”. Trong: *Machine Learning* 3.1 (1988), trang 9–44.
- [4] Christopher J. C. H. Watkins. “Learning from Delayed Rewards”. PhD thesis. Cambridge, U.K.: King’s College, University of Cambridge, 1989.
- [5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, và Martin Riedmiller. “Playing Atari with Deep Reinforcement Learning”. Trong: *arXiv preprint arXiv:1312.5602* (2013).
- [6] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, và các cộng sự. “Human-Level Control Through Deep Reinforcement Learning”. Trong: *Nature* 518.7540 (2015), trang 529–533.
- [7] Richard S. Sutton và Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018.
- [8] Hado Van Hasselt, Arthur Guez, và David Silver. “Deep Reinforcement Learning with Double Q-Learning”. Trong: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 30. 1. 2016.
- [9] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Van Hasselt, Marc Lanctot, và Nando De Freitas. “Dueling Network Architectures for Deep Reinforcement Learning”. Trong: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. 2016, trang 1995–2003.

- [10] Jacopo Panerati, He Zhang, Siqu Zhou, James Xu, Amanda Prorok, và Angela P. Schoellig. “Learning to Fly—A Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-Agent Quadcopter Control”. Trong: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2021.
- [11] Erwin Coumans và Yunfei Bai. *PyBullet Quickstart Guide*. Bullet Physics SDK Documentation. 2016.
- [12] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, và Wojciech Zaremba. “OpenAI Gym”. Trong: *arXiv preprint arXiv:1606.01540* (2016).

PHỤ LỤC A. Phụ lục A

PHỤ LỤC B. Phụ lục B